

THE HAVEN PROTOCOL

DOCUMENT 3

MULTI-PARTY CRYPTOGRAPHIC IDENTITY MATRIX

MCIM v2.0

A PRIVACY-PRESERVING IDENTITY RECONCILIATION LAYER FOR FRAGMENTED HEALTHCARE NETWORKS

Technical Specification

Protocol: The Haven Protocol

Document: 03 of 05

Specification: Multi-Party Cryptographic Identity Matrix

Abbreviation: MCIM

Version: 2.0

Classification: Core Protocol Architecture

Deployment Model: Stateless / Federated / Zero-Trust

Primary Environment: Cross-Facility and Cross-Border Healthcare Interoperability

CONFIDENTIAL INSTITUTIONAL Technical Document. This architecture suite is published openly on GitHub and is licensed strictly under the terms of the GNU Affero General Public License (AGPL-3.0). Unauthorized derivative commercial execution or private re-licensing without an enterprise waiver from the author is strictly prohibited.

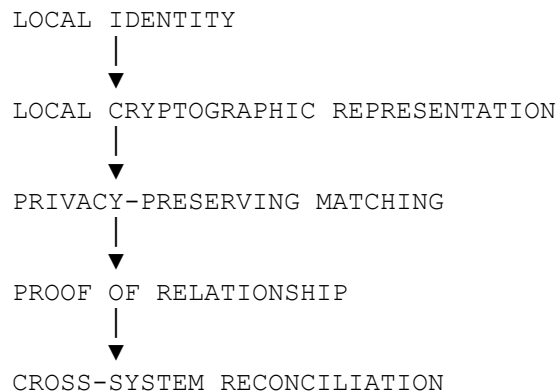
Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

ARCHITECTURAL PURPOSE

The Multi-Party Cryptographic Identity Matrix defines the identity layer through which Haven participating systems can determine whether records originating from different healthcare organizations refer to the same underlying patient **without requiring those organizations to exchange their raw identity databases**.

The MCIM SHALL operate as a privacy-preserving identity coordination mechanism rather than as a centralized patient master database.

Its purpose is to establish:



without requiring:

CENTRAL RAW PATIENT DATABASE

as the mechanism of reconciliation.

CORE SECURITY PRINCIPLE

The protocol SHALL separate:

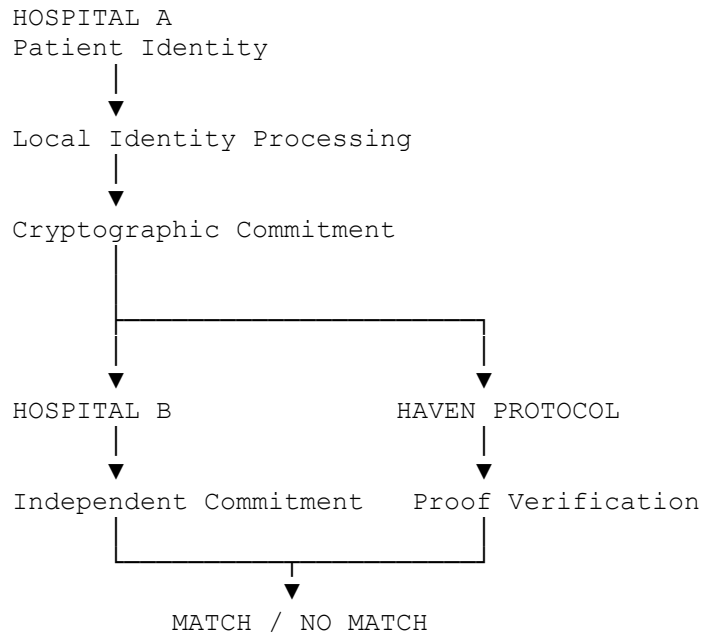
IDENTITY KNOWLEDGE

from:

IDENTITY PROOF

A participating institution may possess the patient's identifying information while Haven receives only the minimum cryptographic material required to establish an authorized relationship between records.

Conceptually:



The protocol SHALL NOT require Hospital A to disclose its complete patient identity record to Hospital B merely to establish whether two records correspond.

ZERO-TRUST IDENTITY MODEL

No participating organization SHALL be trusted to declare:

"These two records belong to the same patient."

without cryptographic evidence satisfying the active identity-matching policy.

Likewise, no central Haven operator SHALL be required to possess the underlying identity attributes merely to verify a valid identity relationship.

The identity architecture therefore establishes:

CLAIM
≠
PROOF

LOCAL MATCH
≠
GLOBAL AUTHORITY

IDENTIFIER
≠
PATIENT IDENTITY

CRYPTOGRAPHIC COMMITMENT
≠
PLAINTEXT IDENTITY

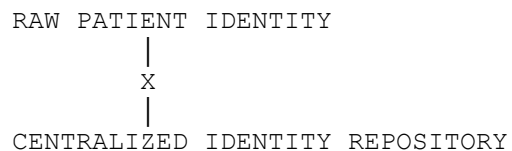
PRIMARY SECURITY OBJECTIVES

The MCIM SHALL provide:

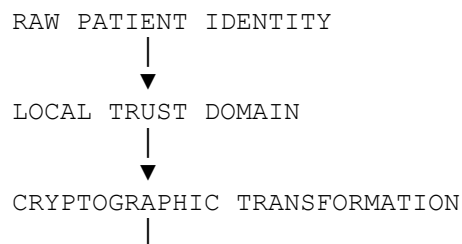
01. Deterministic identity normalization
02. Privacy-preserving identity commitments
03. Cross-party record matching
04. Cryptographic proof of matching conditions
05. Replay resistance
06. Token expiration and revocation
07. Domain separation between organizations
08. Cross-border identity reconciliation
09. Minimal disclosure
10. Identity-link auditability
11. Non-reversible identity representations where cryptographically appropriate
12. Fail-closed handling of ambiguous matches

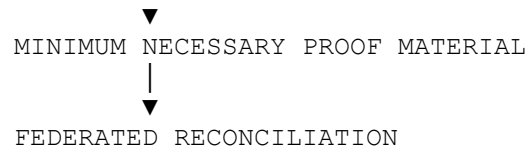
GOVERNING INVARIANT

The identity layer SHALL follow:



and instead:





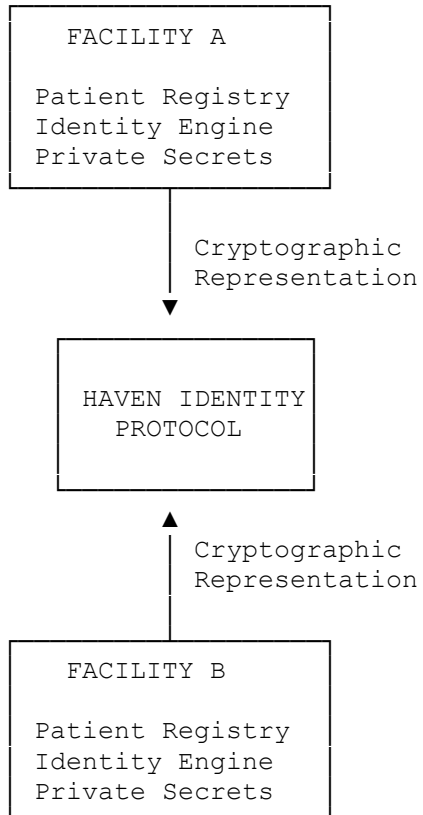
The MCIM SHALL therefore function as an **identity proof layer**, not a centralized identity ownership layer.

MCIM ARCHITECTURAL MODEL

1.1 Identity Trust Domains

Every participating healthcare organization SHALL operate within an independent identity trust domain.

Example:



Facility A SHALL retain control over its local identity attributes.

Facility B SHALL independently retain control over its own identity attributes.

The Haven protocol SHALL coordinate the cryptographic relationship between them without becoming the authoritative owner of either source identity database.

1.2 Identity Representation Layers

The MCIM SHALL distinguish at least four identity layers:

- L0 – Raw Identity
Name, DOB, phone, national identifier, etc.
- L1 – Normalized Identity
Canonically formatted local attributes.
- L2 – Cryptographic Commitment
Deterministic representation generated
from approved identity material.
- L3 – Proof / Match Artifact
Cryptographic evidence that specified
matching conditions have been satisfied.

Raw identity SHOULD remain inside L0.

The protocol's cross-party interoperability layer SHALL operate primarily on L2 and L3.

1.3 No Central Master Patient Identifier

The MCIM SHALL NOT require Haven to maintain a universally exposed plaintext master patient identifier.

Instead, the protocol SHALL derive relationship-specific identifiers or commitments according to:

```
Trust Domain
+
Protocol Version
+
Identity Attributes
+
Secret Material
+
Purpose / Domain Separator
```

This prevents the same cryptographic representation from automatically becoming a universal identifier across unrelated contexts.

1.4 Domain Separation

Identity commitments SHALL use explicit domain separation.

Conceptual construction:

```
COMMITMENT =  
H (  
    DOMAIN_SEPARATOR  
    ||  
    PROTOCOL_VERSION  
    ||  
    TRUST_DOMAIN  
    ||  
    NORMALIZED_IDENTITY  
    ||  
    SECRET  
)
```

Where:

H = approved cryptographic hash / commitment primitive

The implementation SHALL use an approved cryptographic construction rather than an ad-hoc concatenation scheme.

1.5 Deterministic Does Not Mean Globally Linkable

The identity engine SHALL be deterministic within the defined matching domain while minimizing unnecessary cross-domain linkability.

Conceptually:

```
Same identity  
+  
Same trust domain  
+  
Same protocol profile  
+  
Same approved inputs  
↓  
Same commitment
```

while:

```
Same identity
+
Different domain separator
      |
      ▼
Different commitment
```

This distinction is essential to prevent one leaked identifier from becoming a universal cross-system tracking primitive.

1.6 Identity Matching Decision Classes

The MCIM SHALL NOT reduce every comparison to a binary:

MATCH / NO MATCH

The decision engine SHALL support at minimum:

```
MATCH
NO_MATCH
AMBIGUOUS
INSUFFICIENT_EVIDENCE
REVOKED
EXPIRED
INVALID_PROOF
```

Only MATCH satisfying the active policy SHALL authorize identity reconciliation.

AMBIGUOUS and INSUFFICIENT_EVIDENCE SHALL fail closed.

1.7 Privacy Boundary

The identity engine SHALL minimize disclosure at every boundary.

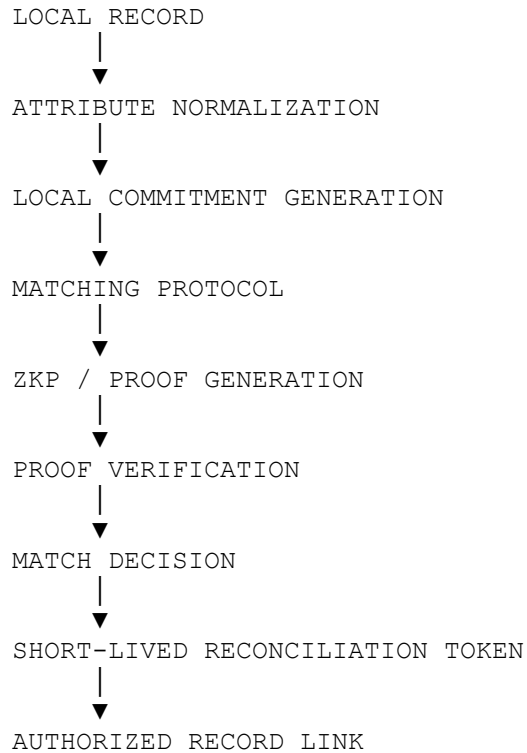
The following SHALL NOT be transmitted merely to perform an ordinary identity-match operation:

```
Complete patient demographic record
Unnecessary clinical history
Complete national-ID record
Raw authentication credentials
Local database primary keys
Private cryptographic secrets
```

Only the identity material required by the active matching protocol SHALL participate.

1.8 Identity Proof Lifecycle

A generalized identity reconciliation transaction SHALL follow:



The reconciliation token SHALL not become a permanent substitute for the patient's underlying identity.

1.9 Identity Authority Separation

The MCIM SHALL separate:

ATTRIBUTE AUTHORITY

from:

MATCHING AUTHORITY

The source facility remains authoritative for its own patient attributes.

The identity protocol determines whether the available cryptographic evidence satisfies the configured reconciliation policy.

This prevents Haven from silently becoming the source of truth for demographic identity.

1.10 Fundamental Privacy Invariant

The MCIM SHALL enforce:

MINIMUM DISCLOSURE
+
PURPOSE LIMITATION
+
DOMAIN SEPARATION
+
CRYPTOGRAPHIC PROOF
+
SHORT-LIVED AUTHORIZATION
=
PRIVACY-PRESERVING IDENTITY RECONCILIATION

The objective is not to claim that privacy risk mathematically becomes zero.

The objective is to remove the architectural requirement for **routine plaintext identity exchange and centralized identity aggregation** from the cross-party reconciliation path.

CRYPTOGRAPHIC FOUNDATION

2.1 Approved Cryptographic Architecture

The MCIM SHALL use established cryptographic primitives rather than proprietary identity-hashing mechanisms.

The baseline architecture SHALL distinguish four cryptographic functions:

MCIM CRYPTOGRAPHIC LAYER	
Hashing	SHA-256 / approved equivalent
Commitment	Keyed commitment / HMAC-based derivation
ZKP	PLONK-family / Groth16-compatible circuit
Signing	Ed25519 / approved asymmetric equivalent

The final primitive selected for production SHALL be recorded in the protocol cryptographic registry.

No implementation SHALL substitute a weaker primitive merely for implementation convenience.

2.2 Cryptographic Separation

The protocol SHALL NOT use the same secret for:

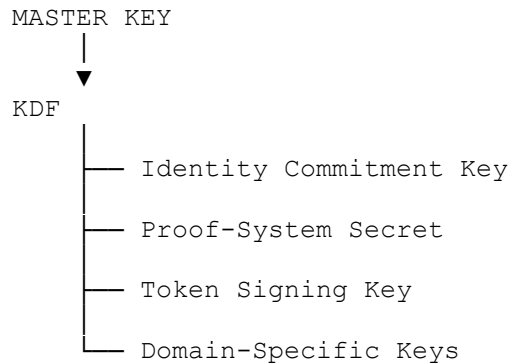
Identity Commitment
Token Signing
Transport Authentication
Database Encryption

Each security function SHALL have independent key material or an independently derived cryptographic context.

2.3 Key Derivation

Where hierarchical derivation is required, keys SHALL be derived using an approved KDF.

Conceptual model:



A production implementation SHALL NOT directly reuse a master secret as an operational identity key.

2.4 Commitment Construction

The preferred commitment mechanism SHALL provide:

Determinism
Domain Separation
Preimage Resistance
Collision Resistance
Secret Binding

Conceptual representation:

```
commitment = HMAC(  
    commitment_key,  
    domain_separator  
    + protocol_version  
    + normalized_identity  
)
```

The exact byte serialization SHALL be canonical and versioned.

The same logical identity SHALL therefore produce a reproducible commitment only within the same defined cryptographic domain.

2.5 Why Plain SHA-256 Is Insufficient for Identity

The protocol SHALL NOT treat:

```
SHA256(name + date_of_birth)
```

as an adequate privacy-preserving identity representation.

Low-entropy demographic attributes may be susceptible to dictionary enumeration.

Therefore:

$$\begin{array}{c} \text{LOW-ENTROPY IDENTITY} \\ + \\ \text{PUBLIC HASH} \\ = \\ \text{ENUMERATION RISK} \end{array}$$

The commitment construction SHALL incorporate protected secret material and domain separation.

2.6 Canonical Serialization

Before cryptographic processing, identity attributes SHALL be serialized deterministically.

Example canonical representation:

```
v=1
domain=facility-A
name=OBINNA|EZE
dob=19990101
sex=M
phone=2348012345678
```

The actual serialization format SHALL be machine-defined rather than manually constructed.

Recommended representation:

```
Canonical JSON
or
Canonical CBOR
```

The protocol SHALL ensure identical logical inputs produce identical canonical bytes.

2.7 Identity Attribute Classification

Attributes SHALL be classified according to their matching strength.

Example:

Attribute Class	Example	Matching Role
Strong	Government-issued identifier	High
Strong	Verified phone number	High
Strong	Verified email	High

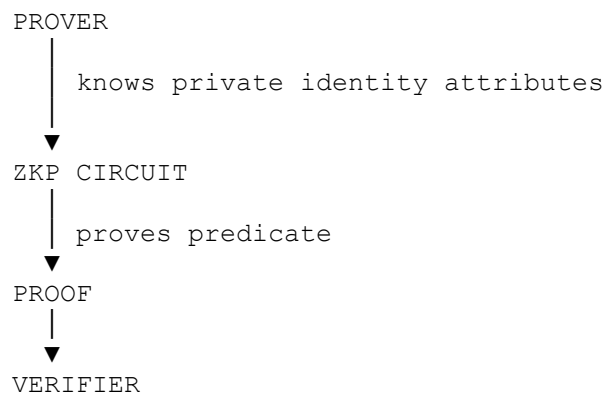
Attribute Class	Example	Matching Role
Medium	Date of birth	Supporting
Medium	Legal surname	Supporting
Medium	Given name	Supporting
Weak	Address	Supporting
Weak	Free-text name variant	Supporting

No single weak attribute SHALL ordinarily establish a cross-party identity match.

2.8 Proof of Knowledge

The ZKP subsystem SHALL allow a participant to prove possession of identity attributes satisfying a matching predicate without revealing the underlying values.

Conceptually:



The verifier receives evidence of the predicate rather than the underlying identity values.

2.9 ZKP Statement

The statement being proven SHALL be explicit.

Example:

"I know a valid identity record whose approved normalized attributes produce a commitment corresponding to the authorized reconciliation domain."

The proof SHALL not disclose:

Patient name
Date of birth
Phone number
Government identifier
Address

unless a separate legal and technical authorization explicitly requires disclosure.

2.10 Public and Private Inputs

The ZKP circuit SHALL distinguish public inputs from private witness values.

Example:

```
PUBLIC INPUTS
  commitment
  domain_separator
  protocol_version
  policy_id

PRIVATE WITNESS
  normalized_name
  date_of_birth
  verified_identifier
  secret
```

The verifier SHALL receive only the inputs required to verify the proof.

2.11 Proof Validity

The proof verifier SHALL return one of:

```
VALID
INVALID
MALFORMED
UNSUPPORTED_VERSION
REVOKED_DOMAIN
```

Only `VALID` proofs satisfying the active policy SHALL proceed.

2.12 Zero-Knowledge Property

The system SHALL be designed so that knowledge of a valid proof does not, by itself, disclose the private witness used to generate that proof.

The security claim SHALL be tied to the selected proof system's formal security assumptions and implementation.

The protocol SHALL therefore avoid the imprecise assertion:

"ZKP makes identity impossible to discover."

The technically correct claim is:

ZKP minimizes the information
required to prove an identity predicate.

DETERMINISTIC PATIENT IDENTITY MATCHING ENGINE

3.1 Matching Objective

The identity engine SHALL determine whether two locally held records satisfy a defined identity equivalence policy.

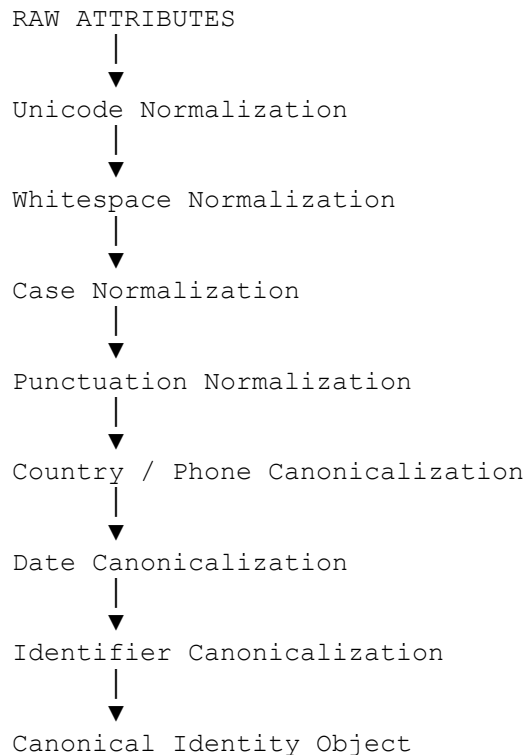
The matching engine SHALL not rely on arbitrary fuzzy similarity alone.

It SHALL use a deterministic sequence of:

```
Normalization
+
Attribute Validation
+
Cryptographic Commitment
+
Policy Evaluation
+
Proof Verification
+
Decision Classification
```

3.2 Identity Normalization Pipeline

Every identity candidate SHALL pass through:



Normalization rules SHALL be versioned.

A change to normalization logic SHALL create a new mapping or identity-engine version.

3.2.1 Regional & Multi-Part Name Normalization

The MCIM Identity Normalization Engine SHALL apply deterministic Unicode and token normalization rules before any identity commitment, keyed derivation, or zero-knowledge proving operation.

3.2.1.1 Compound Surnames

Compound surnames SHALL be normalized by removing internal punctuation characters, including hyphens and apostrophes, and concatenating the resulting surname components into a single contiguous canonical string.

Examples:

```
"Obi-Nasir"      → "obinasir"  
"O'Connor"      → "oconnor"  
"Abdul-Rahman" → "abdulrahman"
```

The normalization operation SHALL occur before generation of the identity cryptographic input stream.

3.2.1.2 Patronymic / Name-Order Variants

Multi-token names SHALL be canonicalized according to the active regional identity policy.

Where the policy identifies a validated primary family identifier, secondary name-token components SHALL be normalized and sorted alphabetically following the primary family identifier.

Canonical representation SHALL preserve token-level boundary spaces using the ASCII space character:

0x20

Conceptually:

```
PRIMARY_FAMILY_IDENTIFIER  
+  
0x20  
+  
SORTED_SECONDARY_TOKEN_1  
+  
0x20  
+
```

SORTED_SECONDARY_TOKEN_2
...

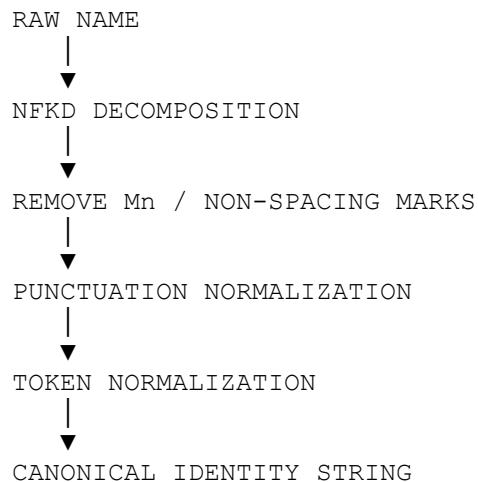
The sorting operation SHALL be deterministic and SHALL use the normalized token representation rather than the original presentation form.

3.2.1.3 Diacritics and Special Tokens

Unicode normalization SHALL use **NFKD decomposition** before cryptographic identity input generation.

Following decomposition, Unicode non-spacing mark characters (M_n) SHALL be removed.

Conceptually:



This ensures that canonically equivalent representations do not produce divergent cryptographic inputs solely because of Unicode encoding differences.

3.2.1.4 Canonicalization Ordering

The normalization pipeline SHALL execute in a deterministic order:

1. Unicode NFKD decomposition
2. Remove non-spacing marks
3. Normalize case
4. Normalize approved punctuation
5. Apply compound-name rules
6. Tokenize
7. Apply regional name-order policy
8. Sort permitted secondary tokens
9. Reconstruct using 0x20 boundaries
10. Validate canonical output
11. Generate K_IDENTITY input stream

No cryptographic commitment SHALL be generated from the raw presentation-layer name.

3.2.1.5 Regional Policy Boundary

Name normalization SHALL remain policy-aware.

The protocol SHALL NOT assume that every jurisdiction uses identical:

- Family-name conventions
- Patronymic structures
- Maternal-name structures
- Compound-name structures
- Name ordering
- Transliteration conventions

Accordingly, the normalization profile SHALL be versioned and associated with the applicable jurisdiction or trust-domain policy.

Example:

```
normalizationProfile:
  jurisdiction = "NG"
  profileVersion = "1.0"
  unicodeForm = "NFKD"
  stripMarks = true
  separator = "0x20"
```

3.2.1.6 Determinism Requirement

For identical normalized input and identical normalization-profile version:

$$N(x)=N(x)N(x)=N(x)$$

The resulting canonical identity representation SHALL be byte-for-byte deterministic.

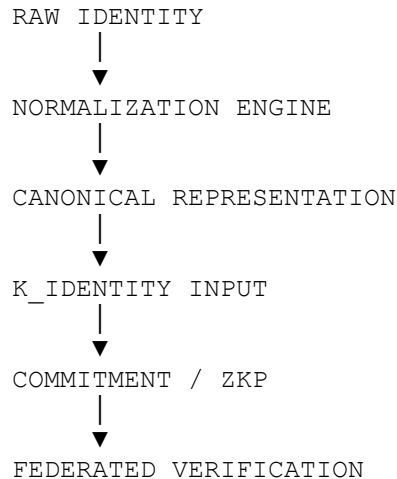
Consequently:

$$KIDENTITY=f(N(x),profileVersion,domainSeparator)K_{\{\text{IDENTITY}\}} = f(N(x), \text{profileVersion}, \text{domainSeparator})$$

A change to normalization rules SHALL result in a new profile version and SHALL NOT silently reinterpret previously generated identity artifacts.

3.2.1.7 Cryptographic Boundary

The final canonical representation SHALL be treated as an intermediate cryptographic input and SHALL not itself be transmitted to an external trust domain unless explicitly authorized by the applicable policy.



This preserves the architectural separation between **identity normalization**, **cryptographic identity construction**, and **cross-domain verification**.

3.3 Name Normalization

Names SHALL NOT be matched using raw string equality.

The normalization engine MAY perform:

- Unicode normalization
- Case folding
- Whitespace collapse
- Punctuation removal
- Approved transliteration
- Known token-order normalization

Example:

"Dr. Eze Obinna"
"OBINNA EZE"
"eze obinna"

may normalize to a common canonical representation where the configured jurisdictional policy permits.

Normalization SHALL NOT silently assume that different names represent the same individual.

3.4 Telephone Normalization

Telephone numbers SHALL be normalized into an approved international representation.

Example:

```
08012345678
  |
  ▼
+2348012345678
```

Country interpretation SHALL be determined from an explicit jurisdictional context rather than guessed from arbitrary input.

3.5 Date Normalization

Dates SHALL be represented using an unambiguous canonical format.

Example:

```
DD/MM/YYYY
  |
  ▼
YYYY-MM-DD
```

Invalid or ambiguous dates SHALL not be automatically corrected.

3.6 Strong Identifier Handling

Government or institutional identifiers SHALL receive special treatment.

The raw identifier SHALL remain within the local trust boundary wherever possible.

The matching representation SHOULD be:

```
HMAC (
    identity_key,
    domain_separator
    +
    canonical_identifier
)
```

rather than:

```
SHA256(canonical_identifier)
```

This reduces straightforward offline enumeration against predictable identifier spaces.

3.7 Deterministic Matching Predicate

A baseline high-assurance policy MAY be expressed as:

```
MATCH =  
    StrongIdentifierMatch  
    AND  
    BirthDateMatch
```

or, where no strong identifier exists:

```
MATCH =  
    VerifiedPhoneMatch  
    AND  
    BirthDateMatch  
    AND  
    NameMatch
```

The exact predicate SHALL be defined by jurisdiction, risk classification, and deployment policy.

3.8 Multi-Attribute Policy

A configurable policy engine MAY evaluate:

```
Identity Score  
├── Strong Identifier  
├── Date of Birth  
├── Name  
├── Phone  
├── Email  
└── Supporting Attribute
```

However, numerical scoring SHALL NOT override mandatory cryptographic or policy conditions.

A score of 0.99 SHALL NOT authorize a match when a required proof is absent.

3.9 Example Deterministic Algorithm

```
def determine_identity_match(candidate_a, candidate_b):  
  
    a = normalize_identity(candidate_a)  
    b = normalize_identity(candidate_b)  
  
    if not required_attributes_valid(a):  
        return "INSUFFICIENT_EVIDENCE"  
  
    if not required_attributes_valid(b):  
        return "INSUFFICIENT_EVIDENCE"
```

```

strong_match = compare_strong_identifier(a, b)
dob_match = constant_time_equal(a.dob_commitment,
                                b.dob_commitment)
name_match = compare_normalized_name(a.name, b.name)
phone_match = constant_time_equal(a.phone_commitment,
                                  b.phone_commitment)

if strong_match and dob_match:
    return "MATCH"

if phone_match and dob_match and name_match:
    return "MATCH"

if (
    dob_match
    and (name_match or phone_match)
):
    return "AMBIGUOUS"

return "NO_MATCH"

```

This algorithm is illustrative.

The production implementation SHALL use a formally versioned matching policy rather than embedding unversioned business logic inside application code.

3.10 Ambiguity Handling

The protocol SHALL deliberately support an `AMBIGUOUS` outcome.

Example:

```

Same Name
+
Same Date of Birth
+
No Strong Identifier
    |
    ▼
AMBIGUOUS

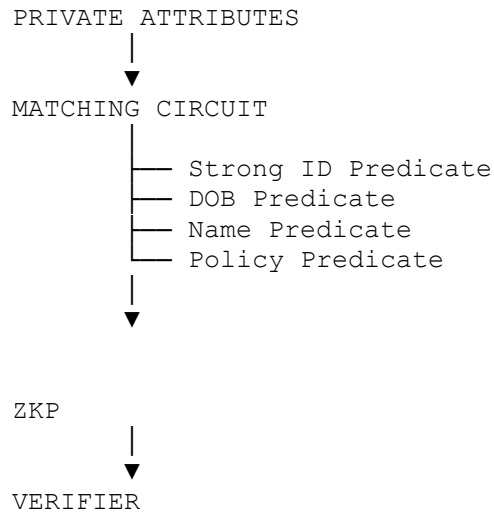
```

The system SHALL NOT convert ambiguity into identity equivalence simply to increase interoperability.

3.11 Match Proof Generation

After a local matching policy determines that the required predicate is satisfied, the participant MAY generate a ZKP demonstrating compliance with that predicate.

Conceptually:



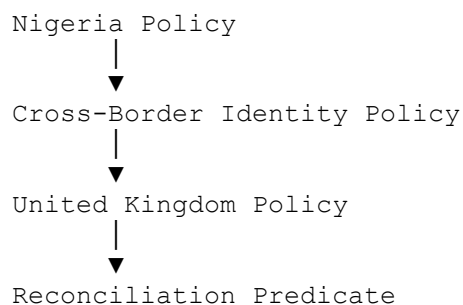
The proof SHALL bind to:

Protocol Version
Policy ID
Trust Domain
Identity Commitment
Expiration Context

3.12 Cross-Border Matching

Cross-border reconciliation SHALL introduce a jurisdiction-specific policy layer.

Example:



The system SHALL NOT assume that an identifier issued in one jurisdiction has identical legal meaning in another.

Jurisdictional differences SHALL therefore be represented explicitly in the identity policy.

3.13 Fundamental Matching Invariant

The MCIM SHALL enforce:

SIMILARITY

≠

IDENTITY

LOCAL ASSERTION

≠

CRYPTOGRAPHIC PROOF

VALID PROOF

≠

UNLIMITED DATA ACCESS

AMBIGUOUS

=

NO AUTOMATIC LINK

The identity engine's primary security objective is therefore not to maximize the number of matched records.

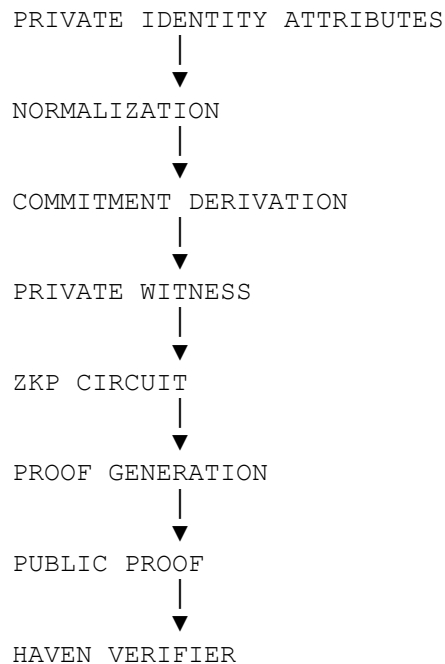
It is to establish **high-confidence, policy-valid identity relationships while minimizing unnecessary disclosure.**

ZERO-KNOWLEDGE PROOF CIRCUIT ARCHITECTURE

4.1 ZKP Execution Model

The MCIM SHALL use zero-knowledge proofs to establish that a participating identity satisfies a predefined reconciliation predicate without requiring the verifier to receive the underlying identity attributes.

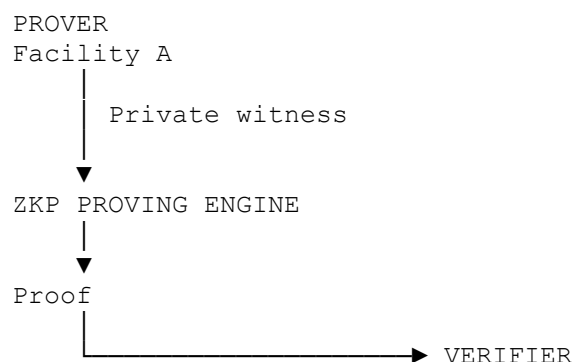
The proof lifecycle SHALL be:



4.2 Prover / Verifier Separation

The participant possessing the underlying identity SHALL operate as the **Prover**.

The Haven identity verification service or authorized participating institution SHALL operate as the **Verifier**.



|
▼
MATCH / REJECT

The Prover SHALL NOT transmit the private witness merely to enable verification.

4.3 Circuit Statement

A baseline circuit SHALL prove a statement equivalent to:

```
"I know a valid set of normalized identity
attributes and secret material which satisfy
the authorized identity-matching predicate
and correspond to the supplied public
commitment."
```

The circuit SHALL bind the proof to the active:

```
Protocol Version
Identity Policy ID
Trust Domain
Commitment
Proof Context
```

4.4 Proving Performance & Edge Latency Targets

The MCIM cryptographic proving layer SHALL be engineered for deterministic performance on constrained edge infrastructure. The following targets SHALL constitute the baseline production performance requirements.

4.4.1 Maximum Proving Latency

The maximum acceptable zero-knowledge proof generation latency shall be:

$T_{\text{prove}} \leq 200 \text{ ms}$

measured from the initiation of the proving operation to the successful generation of the complete proof artifact.

The baseline edge compute environment for this requirement shall be:

```
CPU:          4 vCPUs
Memory:       8 GB RAM
Execution:    Production containerized runtime
```

The proving implementation SHALL NOT rely on specialized GPU acceleration to satisfy the baseline 200 ms requirement unless such acceleration is explicitly declared as an optional deployment optimization.

4.4.2 Verification Latency

Destination-side proof verification SHALL execute within:

$$T_{\text{verify}} \leq 15 \text{ ms}$$

The measurement SHALL cover the cryptographic verification operation itself and SHALL exclude unrelated network transport latency, database retrieval latency, and downstream clinical authorization processing.

The destination gateway SHALL therefore be capable of performing proof verification without introducing material latency into the clinical coordination workflow.

4.4.3 Circuit Constraint Ceiling

The production identity-matching circuit SHALL target a maximum constraint complexity of:

$$C_{\text{circuit}} \leq 2^{17}$$

equivalent to:

$$C_{\text{circuit}} \leq 131,072$$

constraints.

The constraint ceiling SHALL be treated as an architectural resource boundary rather than merely a benchmark target.

Circuit implementations exceeding this ceiling SHALL require explicit architectural review and performance validation before production deployment.

4.4.4 Proving Memory Target

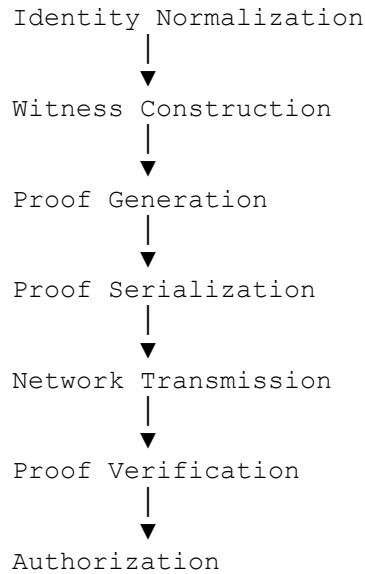
Under the defined 4-vCPU / 8-GB baseline edge environment, steady-state proof generation SHOULD maintain proving-process memory consumption below:

$$M_{\text{prove}} < 512 \text{ MB}$$

This target is intended to preserve sufficient memory headroom for the adapter runtime, database connectors, cryptographic libraries, operating-system processes, telemetry, and circuit-breaker functions executing concurrently on the same edge node.

4.4.5 Performance Measurement Boundary

Performance measurements SHALL distinguish the following stages:



Accordingly:

T_{prove}
=
Proof Generation Time

T_{verify}
=
Cryptographic Verification Time

Network latency SHALL NOT be incorrectly attributed to cryptographic proving or verification latency.

4.4.6 Production Acceptance Thresholds

A circuit implementation SHALL satisfy all baseline thresholds:

Metric	Production Requirement
Maximum proving latency	≤ 200 ms
Maximum verification latency	≤ 15 ms
Maximum circuit complexity	$\leq 131,072$ constraints
Target steady-state proving memory	< 512 MB
Baseline edge CPU	4 vCPUs
Baseline edge memory	8 GB RAM

Failure to satisfy any mandatory threshold SHALL trigger performance optimization, circuit redesign, or formal architectural exception review before production activation.

4.4.7 Performance Regression Control

Every circuit revision SHALL undergo benchmark validation against the currently deployed production version.

The benchmark SHALL record at minimum:

```
Circuit Version
Constraint Count
Witness Size
Proof Size
Median Proving Latency
P95 Proving Latency
Maximum Proving Latency
Median Verification Latency
P95 Verification Latency
Peak Proving Memory
CPU Utilization
```

A circuit revision SHALL NOT be promoted solely because its functional verification tests pass. Cryptographic correctness and production performance SHALL be evaluated independently.

4.4.8 Edge Performance Invariant

The MCIM performance envelope SHALL preserve the following invariant:

$$C_{\text{circuit}} \leq 2^{17} C_{\text{prove}} \quad T_{\text{prove}} \leq 200 \text{ ms} \quad T_{\text{verify}} \leq 15 \text{ ms} \quad T_{\text{verify}} \leq 15 \text{ ms} \quad M_{\text{prove}} < 512 \text{ MB} \quad M_{\text{prove}} < 512 \text{ MB}$$

These thresholds define the baseline computational envelope within which the identity-proofing layer SHALL operate on Haven edge infrastructure.

4.5 Private Witness

The private witness MAY contain:

```
normalized_name
date_of_birth
normalized_phone
normalized_identifier
local_identity_secret
attribute_presence_flags
```

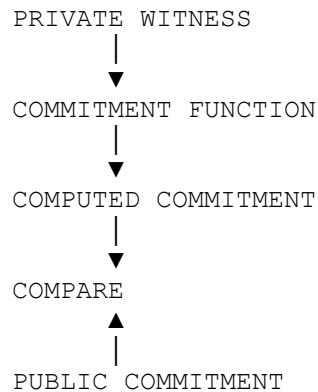
The actual witness composition SHALL depend upon the selected identity policy.

The witness SHALL remain inside the Prover's controlled execution environment.

4.6 Commitment Constraint

The circuit SHALL independently recompute the expected commitment from the private witness.

Conceptually:



If:

```
computed_commitment != public_commitment
```

the proof SHALL fail.

4.7 Predicate Constraint

The circuit SHALL encode the matching policy.

Example:

```
DOB_A == DOB_B  
AND  
HMAC(ID_A) == HMAC(ID_B)
```

Where the selected policy permits such a predicate.

The circuit SHALL NOT prove more than the policy requires.

4.8 Selective Disclosure

Different reconciliation contexts MAY use different proof policies.

Example:

```
POLICY A
Prove same verified phone + DOB

POLICY B
Prove same institutional identifier + DOB

POLICY C
Prove possession of an authorized
identity credential
```

This permits the protocol to avoid using a single universal identity proof for every transaction.

4.9 Nullifiers

To prevent proof replay or unauthorized reuse, the MCIM SHOULD employ a context-specific nullifier or equivalent anti-replay mechanism.

Conceptual construction:

```
NULLIFIER =
H(
    identity_secret
    ||
    domain_separator
    ||
    policy_id
    ||
    context_id
)
```

The nullifier SHALL NOT reveal the underlying patient identity.

Its purpose is to establish that the same proof context has not been improperly reused.

4.10 Proof Context

Every proof SHALL be associated with a bounded context:

```
request_id
purpose
trust_domain
policy_id
issued_at
expires_at
nonce
```

A proof generated for one purpose SHALL not automatically authorize an unrelated purpose.

Example:

```
CARE_CONTINUITY
    ≠
BILLING_RECONCILIATION
    ≠
RESEARCH_ACCESS
```

4.11 Proof Expiration

Proofs SHALL have bounded validity.

Example:

```
issued_at = T0
expires_at = T0 + configured lifetime
```

An expired proof SHALL be rejected.

Long-lived identity proofs SHOULD NOT be used as standing authorization credentials.

4.12 Proof Verification

Conceptual verifier:

```
def verify_identity_proof(proof, public_inputs):

    if proof.protocol_version != ACTIVE_PROTOCOL_VERSION:
        return "UNSUPPORTED_VERSION"

    if proof.expires_at <= current_time():
        return "EXPIRED"

    if not verify_zkp(
        proof.proof,
        public_inputs
    ):
        return "INVALID"

    if nullifier_already_consumed(
        proof.nullifier
    ):
        return "REPLAYED"

    return "VALID"
```

The actual proof library SHALL be selected and security-reviewed before production deployment.

4.13 Trusted Setup Considerations

If the selected proving system requires a trusted setup, the ceremony SHALL be independently auditable and appropriately secured.

Where the protocol's security model favors transparent setup, a proof system with an appropriate transparent or publicly verifiable setup MAY be selected.

The final choice SHALL be documented in the MCIM cryptographic registry.

4.14 Proof-System Versioning

Every proof SHALL be bound to:

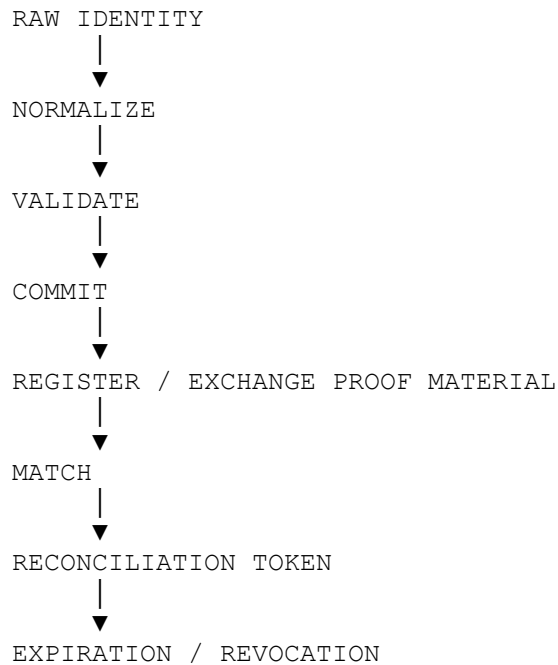
```
proof_system_id  
circuit_version  
protocol_version  
verification_key_version
```

A verifier SHALL reject proofs generated under unsupported circuit or verification-key versions.

IDENTITY COMMITMENT AND MATCHING STATE MACHINE

5.1 Commitment Lifecycle

Identity commitments SHALL follow a controlled lifecycle:



The commitment SHALL not itself grant access to clinical data.

5.2 Commitment Registration

A participating facility MAY register the cryptographic representation necessary for authorized interoperability.

Registration SHALL include sufficient metadata to determine:

- Trust Domain
- Protocol Version
- Identity Policy
- Commitment Version
- Credential Status
- Effective Time
- Expiration

Raw identity attributes SHOULD remain outside the registration payload.

5.3 Commitment Rotation

Commitments SHOULD support controlled rotation.

Rotation may occur because of:

- Secret rotation
- Cryptographic migration
- Policy change
- Domain migration
- Security incident
- Credential revocation

A rotation SHALL NOT automatically invalidate historical audit evidence.

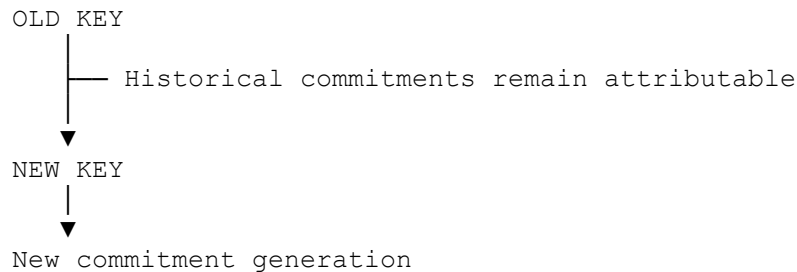
The protocol SHALL preserve sufficient version metadata to distinguish:

- COMMITMENT_V1
- COMMITMENT_V2

5.4 Secret Rotation

Identity commitment keys SHALL be rotatable without requiring plaintext identity data to leave the source trust domain.

Example:



Where historical reconciliation is required, a controlled migration procedure SHALL be used.

5.5 Match Request

A facility requesting identity reconciliation SHALL create a match request containing only the required protocol information.

Example:

```
{
  "requestId": "req-8f42...",
  "policyId": "IDENTITY-STRONG-01",
}
```

```

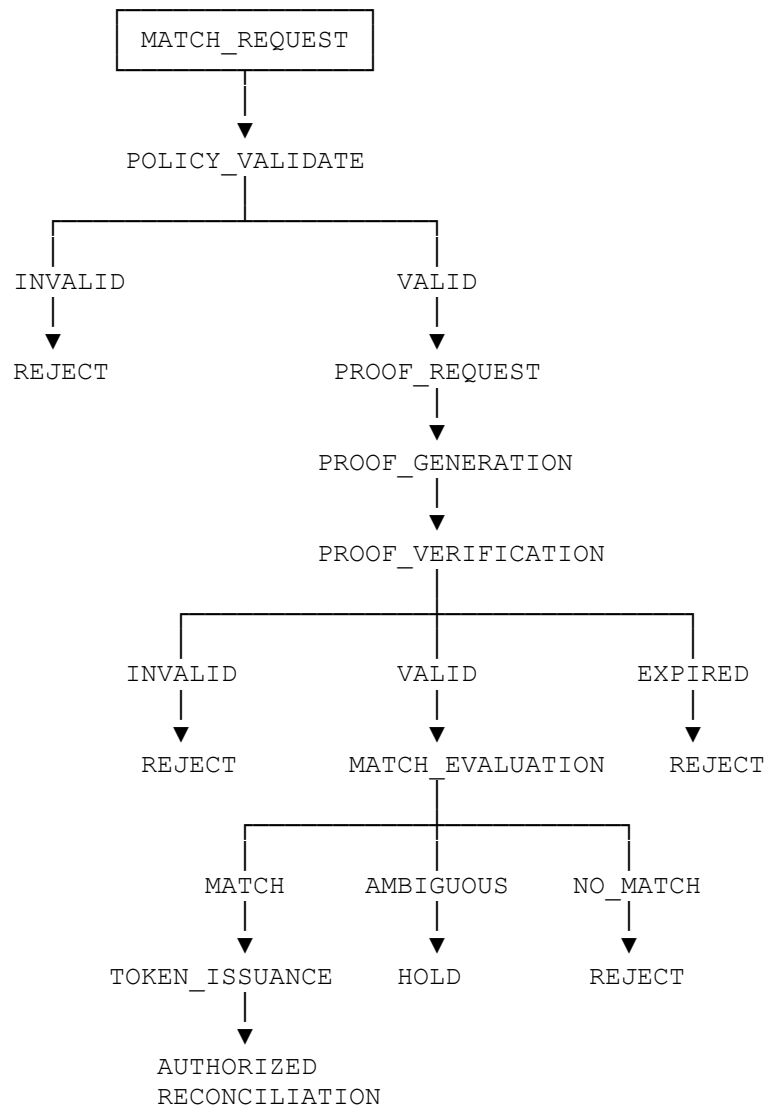
"protocolVersion": "1.0",
"requestingDomain": "FACILITY-A",
"targetDomain": "FACILITY-B",
"purpose": "CARE_CONTINUITY",
"expiresAt": "2026-08-30T13:00:00Z"
}

```

The request SHALL NOT contain unnecessary plaintext identity attributes.

5.6 Identity Matching State Machine

The complete state machine SHALL be:



5.7 Match Authorization

A match SHALL only be established when all required conditions are satisfied:

```
VALID REQUEST
AND
VALID POLICY
AND
VALID PROOF
AND
VALID COMMITMENT
AND
VALID DOMAIN
AND
VALID PURPOSE
AND
NOT EXPIRED
AND
NOT REVOKED
AND
NOT REPLAYED
```

Any failed condition SHALL prevent automatic reconciliation.

5.8 Ambiguous Identity State

The protocol SHALL deliberately preserve an ambiguous state.

Example:

```
Proof Valid
      +
Policy Evidence Insufficient
      |
      ▼
AMBIGUOUS
```

AMBIGUOUS SHALL NOT generate an unrestricted reconciliation token.

A deployment MAY require additional local verification or human adjudication according to its governance policy.

5.9 Identity-Link Token

Following successful proof verification, the protocol MAY issue a short-lived identity-link token.

Conceptual payload:

```
{
  "tokenType": "IDENTITY_LINK",
  "subject": "opaque-subject",
  "requestId": "req-8f42...",
  "policyId": "IDENTITY-STRONG-01",
  "purpose": "CARE_CONTINUITY",
  "issuedAt": "2026-08-30T12:00:00Z",
  "expiresAt": "2026-08-30T12:10:00Z",
  "scope": [
    "RECORD_RECONCILIATION"
  ]
}
```

The token SHALL identify an authorized relationship rather than expose the patient's raw identity.

5.10 Token Scope

Tokens SHALL be least-privilege.

A token issued for:

RECORD_RECONCILIATION

SHALL NOT automatically authorize:

RAW_IDENTITY_DISCLOSURE

or:

UNRESTRICTED_CLINICAL_ACCESS

5.11 Token Revocation

A token SHALL become unusable when:

Token expires
Identity credential revoked
Trust domain revoked
Policy revoked
Purpose cancelled
Security incident invalidates credential

The verifier SHALL check the applicable revocation state before granting the token's protected operation.

5.12 Final Identity State Invariant

The identity engine SHALL enforce:

NO VALID PROOF
↓
NO MATCH AUTHORIZATION

AMBIGUOUS MATCH
↓
NO AUTOMATIC LINK

EXPIRED TOKEN
↓
NO ACCESS

REVOKED CREDENTIAL
↓
NO ACCESS

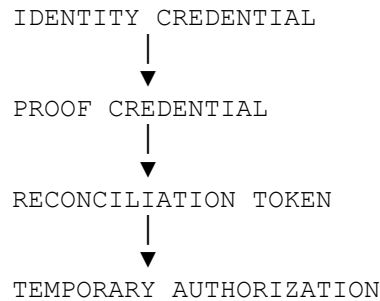
VALID PROOF
↓
LIMITED AUTHORIZATION

The result is a privacy-preserving reconciliation mechanism in which **identity relationships are proven only to the degree required by the authorized policy, for the specific purpose and time window in which the relationship is needed.**

CRYPTOGRAPHIC CREDENTIAL AND TOKEN LIFECYCLE

6.1 Credential Architecture

The MCIM SHALL distinguish between:



These artifacts SHALL have independent purposes and lifecycles.

Possession of one artifact SHALL NOT automatically confer the authority represented by another.

6.2 Credential Classes

The protocol SHALL support at minimum:

DOMAIN CREDENTIAL
Identifies an authorized participating trust domain.

PROVER CREDENTIAL
Authorizes a facility to generate protocol proofs.

VERIFIER CREDENTIAL
Authorizes a component to verify proofs.

RECONCILIATION TOKEN
Authorizes a specific identity-link operation.

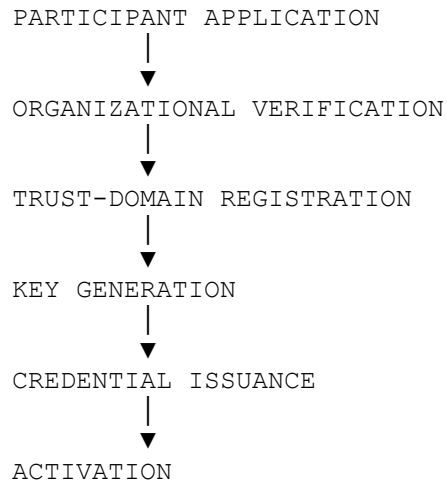
TRANSPORT CREDENTIAL
Authorizes authenticated protocol communication.

Each credential SHALL have a unique cryptographic identity.

6.3 Credential Issuance

Credential issuance SHALL occur through an authenticated enrollment process.

Reference sequence:



Private keys SHALL be generated and retained under the control of the participating trust domain.

The protocol operator SHALL not require possession of participant private keys.

6.4 Key Generation

Private cryptographic material SHOULD be generated using a hardware-backed or protected cryptographic subsystem where available.

Suitable implementations MAY include:

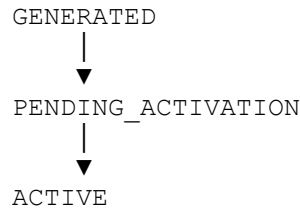
- HSM
- TPM-backed keystore
- Cloud HSM
- Secure enclave
- Hardware-backed signing service

Private identity secrets SHALL not be stored in:

- Source database fields
- Application source code
- Plaintext configuration files
- Git repositories
- Container images
- Ordinary log files

6.5 Credential Activation

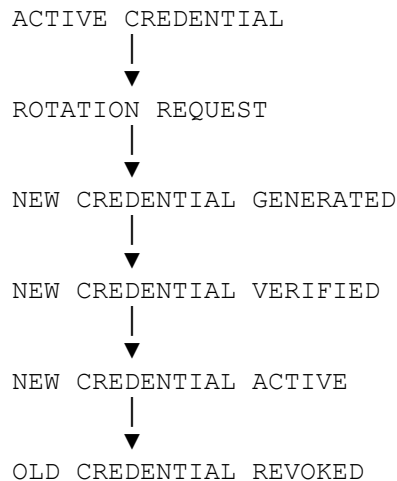
A credential SHALL transition through:



Only an **ACTIVE** credential may participate in production identity reconciliation.

6.6 Credential Rotation

Credentials SHALL have controlled rotation procedures.



The transition SHALL prevent an unnecessary period in which no valid credential exists.

6.7 Credential Revocation

A credential SHALL be revocable independently of the underlying patient identity.

Revocation conditions MAY include:

- Private-key compromise
- Facility compromise
- Unauthorized proof generation
- Contractual termination
- Jurisdictional restriction
- Policy violation
- Credential expiration
- Security incident

The revocation status SHALL be checked according to the applicable verification policy.

6.8 Patient Identity Secret Rotation

Patient-specific cryptographic secrets SHOULD be rotated independently from institutional transport credentials.

This provides separation between:

FACILITY COMPROMISE

and:

LONG-TERM PATIENT LINKABILITY

Where rotation occurs, historical audit records SHALL preserve their original cryptographic version metadata.

6.9 Token Issuance

An identity-link token SHALL only be issued after:

Request Validated
+
Policy Validated
+
Proof Validated
+
Commitment Validated
+
Credential Active
+
Purpose Authorized
+
Token Context Valid

The token issuer SHALL generate a unique token identifier and bounded expiration.

6.10 Token Construction

A production token MAY use a signed compact representation.

Conceptually:

```
TOKEN =  
{  
    token_id,  
    subject_reference,  
    requesting_domain,  
    target_domain,  
    policy_id,  
    purpose,  
    scope,  
    issued_at,  
    expires_at,  
    nonce  
}
```

The token SHALL be integrity-protected by an approved signing mechanism.

6.11 Token Lifetime

Token lifetime SHALL be deliberately short.

Example:

```
Issued:    12:00:00  
Expires:   12:10:00  
Lifetime:  10 minutes
```

The exact duration SHALL depend on the sensitivity of the authorized operation.

Long-lived identity-link tokens SHOULD NOT be used for ordinary care-continuity reconciliation.

6.12 Token Audience

Tokens SHALL identify their intended audience.

Example:

```
aud = FACILITY-B-RECONCILIATION-SERVICE
```

A token presented to an unrelated service SHALL be rejected.

6.13 Token Purpose Binding

The token SHALL be bound to a specific purpose.

Example:

```
purpose = CARE_CONTINUITY
```

The same token SHALL not automatically authorize:

```
RESEARCH  
MARKETING  
IDENTITY_EXPORT  
ADMINISTRATIVE_DISCLOSURE
```

6.14 Token Revocation and Expiration State

The verifier SHALL distinguish:

```
ACTIVE  
EXPIRED  
REVOKED  
MALFORMED  
WRONG_AUDIENCE  
WRONG_PURPOSE  
WRONG_SCOPE  
REPLAYED
```

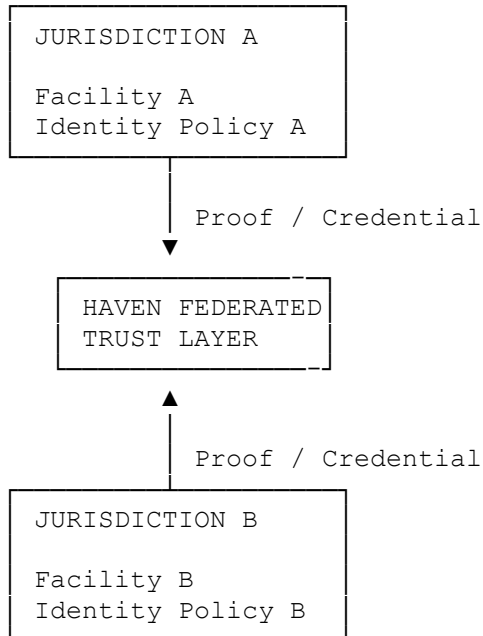
Only `ACTIVE` tokens satisfying all contextual constraints SHALL authorize the protected operation.

CROSS-BORDER TRUST FEDERATION

7.1 Federation Model

Cross-border identity reconciliation SHALL use federated trust rather than assuming a single global identity authority.

Reference topology:



The federation layer SHALL establish whether one trust domain is authorized to interact with another.

7.2 Jurisdictional Policy Registry

Every cross-border relationship SHALL reference an explicit policy.

Example:

```
{
  "policyId": "NG-GB-CARE-01",
  "sourceJurisdiction": "NG",
  "targetJurisdiction": "GB",
  "purpose": "CARE_CONTINUITY",
  "identityPolicy": "STRONG_MATCH_01",
  "proofSystem": "ZKP-V1",
  "tokenLifetimeSeconds": 600,
  "active": true
}
```

The policy registry SHALL be versioned and auditable.

7.3 Trust Establishment

A cross-border trust relationship SHALL require:

```
Recognized Trust Domain
+
Approved Credential
+
Compatible Protocol Version
+
Compatible Proof System
+
Active Jurisdictional Policy
```

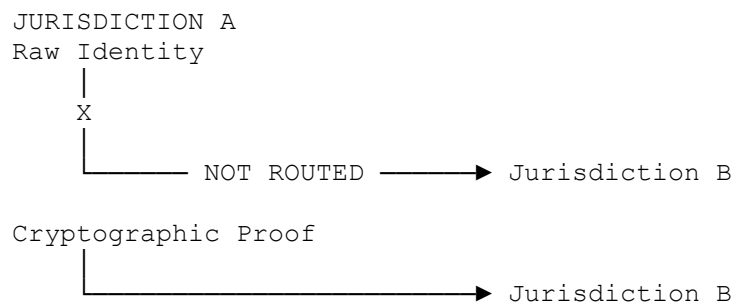
A cryptographically valid proof from an unrecognized trust domain SHALL not automatically authorize reconciliation.

7.4 Data Localization

Where a jurisdiction requires data localization, the MCIM SHALL allow the underlying identity data to remain within the source jurisdiction.

The cross-border protocol SHALL exchange only the cryptographic artifacts required by the approved policy.

Conceptually:



7.5 Jurisdictional Attribute Restrictions

Different jurisdictions MAY require different identity attributes.

Therefore:

POLICY A
Strong Identifier + DOB

POLICY B
Verified Credential + DOB

POLICY C
Institutional Identifier + Local Credential

The MCIM SHALL not assume that one universal identity predicate is legally or operationally appropriate everywhere.

7.6 Purpose Limitation

Cross-border identity reconciliation SHALL be purpose-bound.

Examples:

Emergency Care
Planned Care
Referral
Medication Continuity
Laboratory Continuity
Insurance Verification

A proof authorized for one purpose SHALL not silently authorize another.

7.7 Emergency Mode

A deployment MAY define an emergency identity-reconciliation policy.

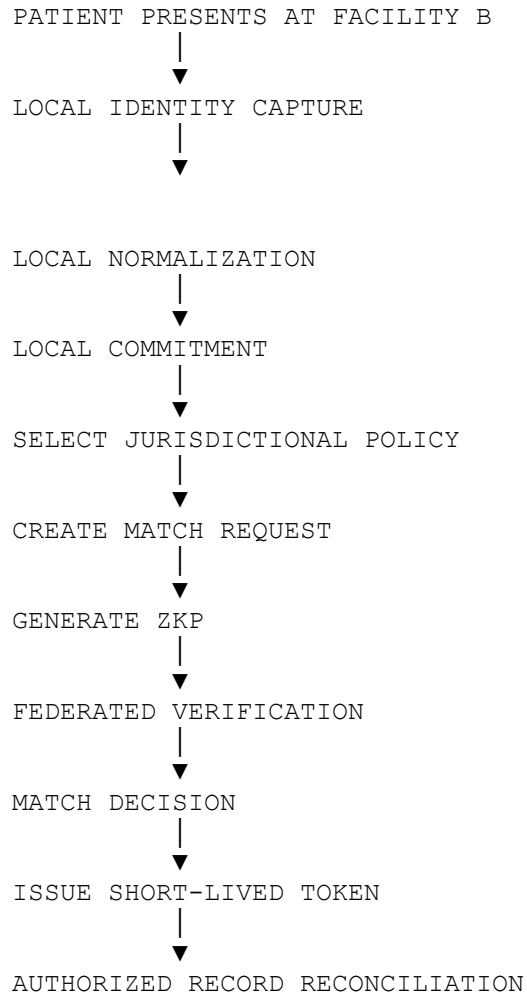
Emergency mode SHALL still maintain:

Authentication
Auditability
Purpose Binding
Minimum Disclosure
Post-event Review

Emergency status SHALL not become a universal bypass mechanism.

7.8 Cross-Border Match Sequence

The canonical cross-border sequence SHALL be:



7.9 Cross-Border Failure States

The following conditions SHALL fail closed:

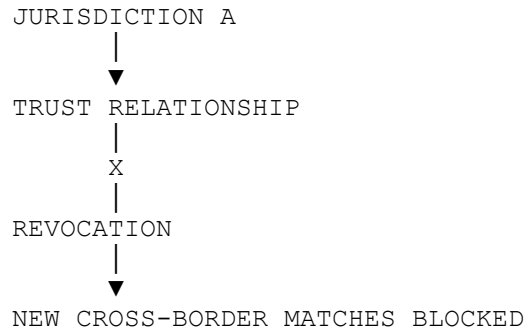
- Unknown Jurisdiction
- Unknown Trust Domain
- Expired Policy
- Invalid Credential
- Invalid Proof
- Conflicting Identity Evidence
- Expired Token
- Revoked Credential
- Unsupported Protocol Version
- Unsupported Proof System

The system SHALL not downgrade to plaintext identity exchange merely because cryptographic reconciliation failed.

7.10 Federation Revocation

A trust relationship SHALL be independently revocable.

Example:



Historical audit records SHALL remain verifiable according to their original cryptographic context.

7.11 Federation Security Invariant

The MCIM SHALL enforce:

VALID IDENTITY PROOF
≠
UNIVERSAL ACCESS

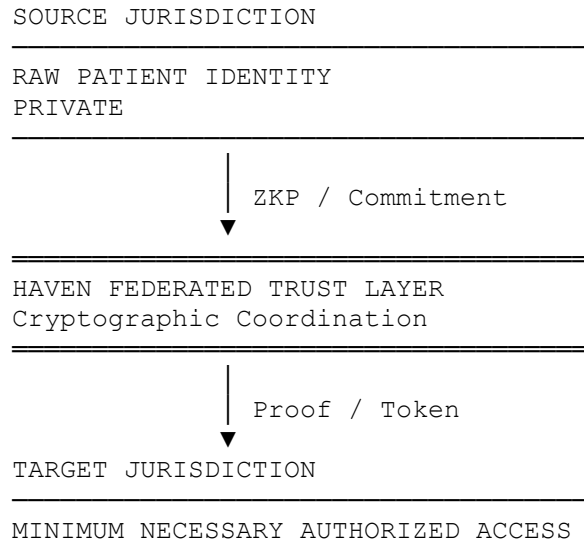
VALID FACILITY
≠
UNIVERSAL TRUST

CROSS-BORDER REQUEST
≠
AUTOMATIC DISCLOSURE

FAILED ZKP
≠
PLAINTEXT FALLBACK

7.12 Privacy Boundary

The final cross-border privacy boundary SHALL therefore be:



This architecture does not claim that regulatory compliance is automatically achieved merely by deploying cryptography.

Instead, it establishes a technical control boundary in which **cross-border reconciliation can occur without making routine plaintext identity replication a prerequisite for interoperability**.

DETERMINISTIC IDENTITY MATCHING ENGINE

8.1 Matching Architecture

The MCIM identity engine SHALL separate identity matching into four independent computational stages:

IDENTITY MATCH ENGINE	
01	NORMALIZATION Canonicalize source attributes
02	DETERMINISTIC COMPARISON Evaluate approved predicates
03	CRYPTOGRAPHIC VERIFICATION Verify commitment / ZKP
04	POLICY DECISION MATCH / AMBIGUOUS / NO_MATCH

No individual stage SHALL independently authorize a cross-party identity link.

8.2 Canonical Identity Object

After normalization, the source identity SHALL be represented internally as a canonical object.

Example:

```
{
  "identityVersion": "1",
  "jurisdiction": "NG",
  "name": {
    "family": "EZE",
    "given": "OBINNA"
  },
  "dateOfBirth": "1999-01-01",
  "phone": "+2348012345678",
  "identifier": {
    "type": "INSTITUTIONAL",
    "value": "LOCAL-REFERENCE"
  }
}
```

The raw canonical object SHALL remain inside the local trust boundary unless an active policy explicitly permits an attribute to leave that boundary.

8.3 Deterministic Attribute Functions

Each attribute SHALL have a defined comparison function.

```
NAME
    Canonical equality / approved alias mapping

DATE OF BIRTH
    Exact canonical equality

PHONE
    Canonical international equality

IDENTIFIER
    Cryptographic commitment equality

EMAIL
    Canonicalized equality

ADDRESS
    Supporting attribute only
```

No undefined comparison behavior SHALL be permitted.

8.4 Strong Match Predicate

The preferred high-assurance predicate SHALL use a strong identifier where available.

Conceptual rule:

```
STRONG_IDENTIFIER_MATCH
    AND
DATE_OF_BIRTH_MATCH
    AND
POLICY_VALID
    AND
PROOF_VALID
    =
MATCH
```

The strong identifier SHALL ordinarily be represented cryptographically rather than transmitted in plaintext.

8.5 Alternative High-Assurance Predicate

Where a strong identifier is unavailable, the policy MAY require multiple independent attributes:

```
VERIFIED_PHONE_MATCH
      AND
DATE_OF_BIRTH_MATCH
      AND
NAME_MATCH
      AND
PROOF_VALID
      =
MATCH
```

The policy SHALL specify which attributes qualify as sufficiently independent for the deployment.

8.6 Attribute Independence

The engine SHALL distinguish between attributes that provide genuinely independent evidence and attributes that are merely different representations of the same source.

For example:

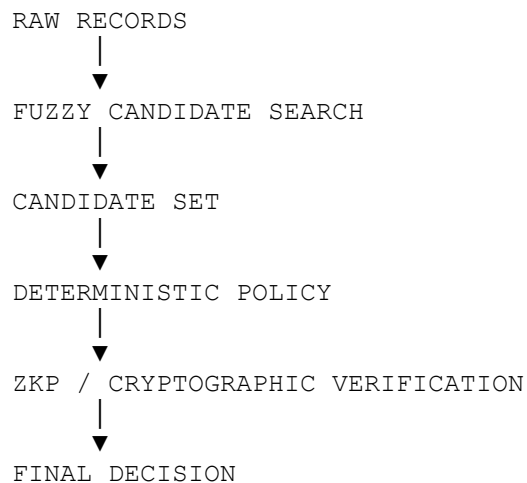
```
Phone
+
Phone formatted differently
```

SHALL NOT be counted as two independent identity signals.

8.7 Fuzzy Matching Containment

Fuzzy matching MAY be used for **candidate discovery**, but SHALL NOT independently establish identity equivalence.

Example:



A high similarity score SHALL therefore be treated as candidate evidence, not authorization.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

8.8 Similarity Score Constraint

Example:

```
if similarity_score >= CANDIDATE_THRESHOLD:
    candidate = True
else:
    candidate = False
```

This result SHALL never be equivalent to:

```
identity_match = True
```

The final match decision SHALL require the active policy predicate.

8.9 Collision Handling

A collision occurs when multiple records satisfy the same preliminary identity conditions.

Example:

```
NAME = JOHN DOE
DOB  = 1980-01-01
```

```
├── RECORD A
├── RECORD B
└── RECORD C
```

The engine SHALL classify this condition as ambiguous unless additional authorized evidence resolves it.

8.10 False-Positive Protection

The MCIM SHALL prioritize preventing false-positive identity links in high-risk clinical workflows.

A false positive can cause:

```
Wrong patient history
Wrong medication context
Wrong laboratory history
Wrong allergy information
Wrong billing relationship
Wrong clinical decision support
```

Accordingly:

AMBIGUOUS

↓

NO AUTOMATIC LINK

8.11 False-Negative Protection

The system SHALL also detect conditions likely to produce false negatives.

Examples:

Name transliteration
Marriage / legal name change
Missing phone
Changed phone
Cross-border formatting
Legacy identifier migration
Incomplete demographic record

These conditions MAY trigger an additional authorized matching path.

They SHALL NOT trigger plaintext identity export by default.

8.12 Match Confidence Classes

The protocol SHALL support policy-defined confidence classes:

CLASS A
High-assurance deterministic match

CLASS B
Strong match with approved secondary evidence

CLASS C
Candidate / requires additional evidence

CLASS D
Ambiguous

CLASS E
No match

Only authorized classes SHALL generate reconciliation tokens.

8.13 Deterministic Match Function

A conceptual production function:

```
def evaluate_identity_policy(a, b, policy):  
    if not policy.is_active:  
        return "REJECT"  
  
    if not valid_required_attributes(a, policy):  
        return "INSUFFICIENT_EVIDENCE"  
  
    if not valid_required_attributes(b, policy):  
        return "INSUFFICIENT_EVIDENCE"  
  
    if policy.requires_strong_identifier:  
        if not constant_time_equal(  
            a.identifier_commitment,  
            b.identifier_commitment  
        ):  
            return "NO_MATCH"  
  
    if policy.requires_date_of_birth:  
        if a.date_of_birth_commitment != \b.date_of_birth_commitment:  
            return "NO_MATCH"  
  
    if policy.requires_name:  
        if not deterministic_name_match(  
            a.normalized_name,  
            b.normalized_name  
        ):  
            return "NO_MATCH"  
  
    return "MATCH"
```

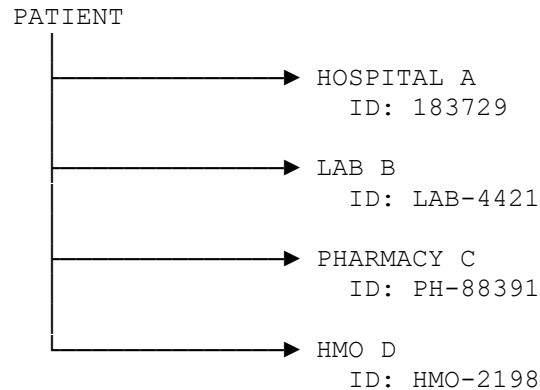
The production implementation SHALL additionally verify the associated ZKP, credentials, policy version, token context, and revocation state before authorization.

FORENSIC IDENTITY-CRISIS ELIMINATION MODEL

9.1 The Multi-Party Identity Crisis

Fragmented healthcare systems commonly maintain independent representations of the same patient.

Conceptually:



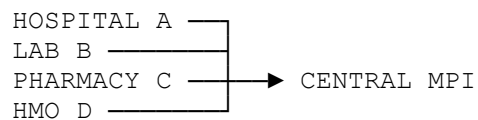
The identifiers are locally meaningful but globally disconnected.

The resulting problem is:

LOCAL IDENTITIES
+
NO SHARED TRUST LAYER
+
NO DETERMINISTIC RECONCILIATION
=
IDENTITY FRAGMENTATION

9.2 Conventional Central-Master Approach

A conventional response is to construct a centralized master patient index.



This creates a new concentration point for:

Identity Data
Linkage Metadata
Access Relationships
Clinical Context
Breach Impact

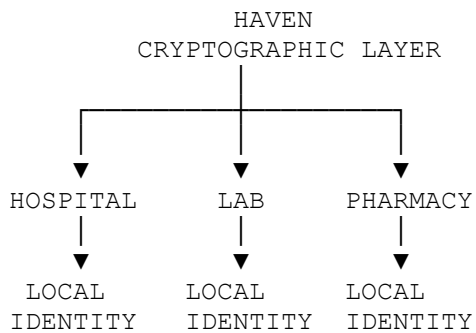
Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

Operational Authority

The MCIM takes a different architectural approach.

9.3 Haven Identity Matrix

The MCIM replaces centralized identity accumulation with cryptographic reconciliation.



The source organizations retain their underlying identity systems.

Haven coordinates the proof relationship.

9.4 Crisis Dimension 1 — Identifier Fragmentation

Problem:

One Patient
↓
Many Local IDs

MCIM response:

Many Local IDs
↓
Deterministic Identity Commitments
↓
Policy-Verified Relationship

The protocol therefore creates a cryptographic relationship without requiring all parties to adopt the same underlying database identifier.

9.5 Crisis Dimension 2 — Duplicate Patient Records

Problem:

Facility A
Patient #1001

Facility B
Patient #7732

The records may represent the same person.

MCIM response:

```
LOCAL IDENTITIES
    ↓
NORMALIZATION
    ↓
COMMITMENTS
    ↓
MATCHING PREDICATE
    ↓
ZKP
    ↓
VERIFIED RELATIONSHIP
```

The relationship is established by evidence rather than by manual identifier synchronization.

9.6 Crisis Dimension 3 — Raw Identity Exchange

Conventional interoperability may require:

```
Name
DOB
Phone
Address
Identifier
    |
    ▼
REMOTE SYSTEM
```

MCIM instead permits:

```
PRIVATE ATTRIBUTES
    |
    ▼
LOCAL PROOF GENERATION
    |
    ▼
ZKP
    |
    ▼
```

REMOTE VERIFICATION

The verifier can establish the required predicate without routinely receiving the complete underlying identity record.

9.7 Crisis Dimension 4 — Cross-Border Identity Mismatch

Different jurisdictions may use:

Different identifiers
Different naming conventions
Different date formats
Different registries
Different institutional credentials
Different privacy requirements

MCIM addresses this through:

JURISDICTIONAL POLICY
+
DOMAIN SEPARATION
+
CANONICAL NORMALIZATION
+
PROOF VERIFICATION

The protocol does not require one jurisdiction to surrender control of its local identity authority.

9.8 Crisis Dimension 5 — Central Identity Honeypot

A centralized repository creates a potentially high-value target.

MCIM reduces this architectural concentration by ensuring that:

RAW IDENTITY

remains distributed across the participating trust domains.

Haven's coordination layer therefore does not need to become a universal plaintext identity warehouse.

9.9 Crisis Dimension 6 — Unauthorized Linkability

A universal identifier can allow unrelated organizations to correlate activity.

MCIM addresses this using:

Domain Separation
+
Purpose Binding
+
Context-Specific Nullifiers
+
Short-Lived Tokens

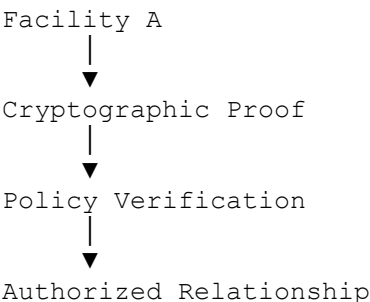
Consequently, a reconciliation artifact generated for one authorized context SHOULD NOT automatically become a universal tracking identifier.

9.10 Crisis Dimension 7 — Identity Assertion Without Evidence

Traditional federation may rely heavily upon:

"Facility A says this is Patient X."

MCIM changes the trust model to:



The protocol therefore separates an organization's assertion from independently verifiable cryptographic evidence.

9.11 Forensic Control Matrix

Identity Failure	Conventional Exposure	MCIM Control
Local ID fragmentation	Manual reconciliation	Cryptographic matching
Duplicate records	Central MPI / manual review	Deterministic policy
Raw identity exchange	Broad disclosure	ZKP / commitment
Cross-border mismatch	Registry incompatibility	Federated policy
Central identity honeypot	Concentrated breach impact	Distributed identity custody
Universal tracking	Global identifier	Domain separation
Credential compromise	Persistent access	Revocation + expiration
Proof replay	Reused authorization	Context + nonce / nullifier
Ambiguous identity	Forced match	Fail-closed ambiguity

Identity Failure

Unauthorized access

Conventional Exposure

Broad federation

MCIM Control

Purpose-bound token

9.12 Architectural Conclusion

The MCIM does **not** claim that identity risk literally becomes mathematically zero.

Instead, it removes several structural causes of multi-party identity exposure:

CENTRALIZED PLAINTEXT IDENTITY DEPENDENCY



REMOVED

ROUTINE RAW IDENTITY EXCHANGE



MINIMIZED

UNIVERSAL IDENTIFIER DEPENDENCY



REDUCED

UNVERIFIED MATCH ASSERTIONS



REPLACED

LONG-LIVED RECONCILIATION AUTHORIZATION



REPLACED

The resulting architecture converts the identity problem from:

"How do we move everyone's patient identities
into one place?"

to:

"How do we prove only the identity relationship
that is necessary, to the party that is authorized,
for the purpose that is permitted, for only as long
as that authorization remains valid?"

That is the fundamental security transformation implemented by the Multi-Party Cryptographic Identity Matrix.

IDENTITY-LINK TOKEN STATE MACHINE

10.1 Token Authorization Boundary

An MCIM reconciliation token SHALL represent a narrowly scoped cryptographic authorization.

It SHALL NOT function as:

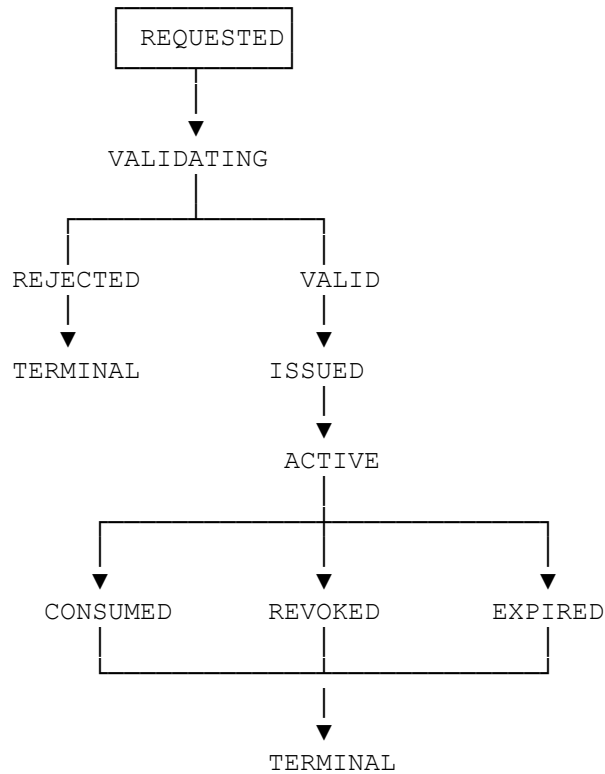
Patient Identity
Master Patient Identifier
Permanent Clinical Credential
Database Credential
Universal Access Token

The token SHALL instead represent:

AUTHORIZED RELATIONSHIP
+
AUTHORIZED PURPOSE
+
AUTHORIZED SCOPE
+
AUTHORIZED TIME WINDOW

10.2 Token State Model

Every reconciliation token SHALL transition through a controlled lifecycle:



A terminal token SHALL NOT return to **ACTIVE**.

10.3 Token Request

A requesting service SHALL establish:

```
request_id  
requesting_domain  
target_domain  
purpose  
policy_id  
requested_scope  
requested_duration  
nonce
```

The request SHALL be authenticated before proof evaluation.

10.4 Policy Evaluation

The token issuer SHALL verify:

```
Trust Domain = ACTIVE
Policy = ACTIVE
Purpose = AUTHORIZED
Scope = PERMITTED
Credential = ACTIVE
Proof = VALID
Context = VALID
```

Failure of any mandatory condition SHALL terminate issuance.

10.5 Token Issuance

After successful verification:

```
VALIDATED REQUEST
      +
VALID PROOF
      +
ACTIVE CREDENTIAL
      +
AUTHORIZED POLICY
      |
      ▼
TOKEN ISSUANCE
```

The token SHALL receive:

```
Unique Token ID
Issue Timestamp
Expiration Timestamp
Audience
Subject Reference
Policy ID
Purpose
Scope
Nonce
Credential Version
```

10.6 Token Presentation

The consuming service SHALL verify the token before executing the protected operation.

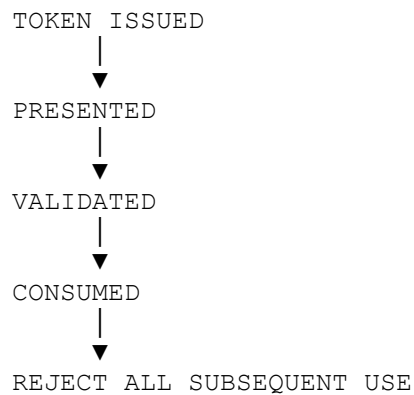
```
def authorize_token(token, context):  
    if token.expires_at <= now():  
        return "EXPIRED"  
  
    if token.audience != context.audience:  
        return "WRONG_AUDIENCE"  
  
    if token.purpose != context.purpose:  
        return "WRONG_PURPOSE"  
  
    if not scope_permitted(  
        token.scope,  
        context.operation  
    ):  
        return "WRONG_SCOPE"  
  
    if is_revoked(token.token_id):  
        return "REVOKED"  
  
    if nonce_consumed(token.nonce):  
        return "REPLAYED"  
  
    return "AUTHORIZED"
```

The consuming service SHALL perform authorization before exposing protected records.

10.7 Single-Use Operations

For high-risk operations, tokens SHOULD be single-use.

Example:



This substantially reduces replay exposure for sensitive identity-link operations.

10.8 Context Binding

A token SHALL be bound to the transaction context for which it was issued.

Conceptual binding:

```
context_hash =  
H(  
    request_id  
    ||  
    purpose  
    ||  
    requesting_domain  
    ||  
    target_domain  
    ||  
    policy_id  
)
```

The consuming service SHALL reject a token whose context does not match the active request.

10.9 Replay Resistance

Replay protection SHALL use multiple controls:

```
Short expiration  
+  
Unique token ID  
+  
Nonce  
+  
Context binding  
+  
Optional single-use consumption  
+  
Credential status verification
```

No single control SHALL be treated as the complete replay-defense mechanism.

10.10 Expiration

Expiration SHALL be enforced independently by the verifier.

A token with:

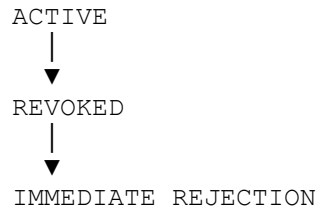
```
expires_at < current_time
```

SHALL be rejected even if:

```
proof = VALID
```

10.11 Revocation

Revocation SHALL take precedence over expiration state.



Revocation MAY occur because of:

- Credential compromise
- Facility compromise
- Policy withdrawal
- Identity correction
- Unauthorized activity
- Jurisdictional restriction
- Security incident

10.12 Token Scope Enforcement

The scope SHALL be evaluated as an allow-list.

Example:

```
{
  "scope": [
    "IDENTITY_RECONCILIATION",
    "RECORD_DISCOVERY"
  ]
}
```

The token SHALL not authorize operations outside those explicit permissions.

10.13 Token Non-Transitivity

A service receiving an MCIM token SHALL not automatically delegate it to another service.

Delegation SHALL require an explicitly authorized protocol mechanism.

Therefore:

```
TOKEN A
  ↓
SERVICE A
  X
SERVICE B
```

unless delegation is expressly permitted by policy.

10.14 Token Audit Record

The protocol SHOULD retain an auditable event record containing:

```
token_id  
request_id  
policy_id  
issuer  
audience  
purpose  
scope  
issued_at  
expires_at  
status  
revocation_reference
```

The audit record SHOULD avoid storing raw patient identity values.

KEY COMPROMISE, REVOCATION AND FORENSIC CONTAINMENT

11.1 Compromise Model

The MCIM SHALL assume that cryptographic credentials can eventually be compromised.

The architecture SHALL therefore be designed around:

```
DETECT
  ↓
CONTAIN
  ↓
REVOKE
  ↓
ROTATE
  ↓
REVALIDATE
  ↓
RECOVER
```

Security SHALL not depend upon the assumption that private keys are permanently safe.

11.2 Compromise Classes

The protocol SHALL distinguish:

```
CLASS 1
Transport credential compromise

CLASS 2
Facility signing-key compromise

CLASS 3
Identity commitment-key compromise

CLASS 4
Proof-generation credential compromise

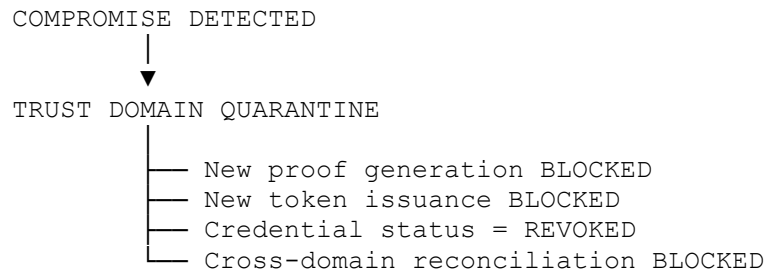
CLASS 5
Verifier credential compromise

CLASS 6
Host or adapter compromise
```

Each class SHALL have a defined response procedure.

11.3 Immediate Containment

When compromise is suspected:

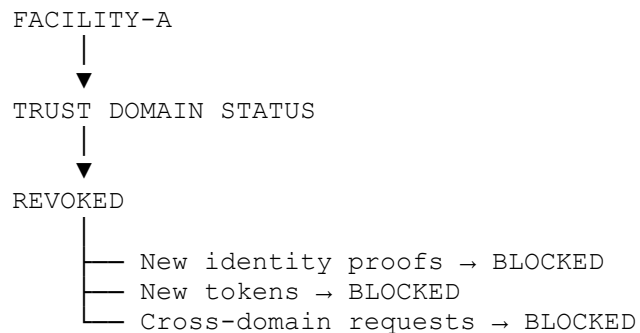


The containment mechanism SHALL operate independently of the compromised application where feasible.

11.4 Domain-Level Revocation

A compromised trust domain MAY be revoked at the federation layer.

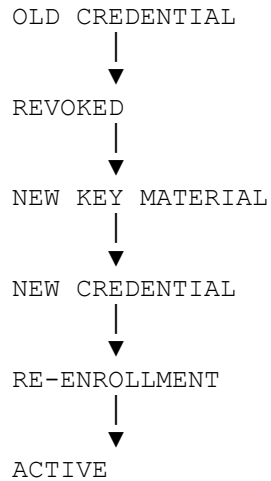
Example:



This permits containment without globally disabling unrelated participants.

11.5 Credential Rotation

Following containment:



The new credential SHALL receive a new version identifier.

11.6 Commitment-Key Compromise

Commitment-key compromise requires special handling because it may increase the ability to test identity hypotheses.

The response SHALL include:

1. Revoke affected commitment key
2. Establish new commitment key
3. Version new commitments
4. Assess exposure window
5. Identify affected trust domains
6. Reconcile historical proofs where necessary
7. Review audit records
8. Determine whether additional notification or regulatory procedures are required

11.7 Historical Proof Integrity

Revoking a credential SHALL not retroactively erase the existence of a previously valid event.

The protocol SHALL distinguish:

EVENT VALID AT TIME T

from:

CREDENTIAL VALID TODAY

Historical audit verification SHALL therefore include the credential and policy state applicable at the original event time.

11.8 Cryptographic Versioning

Every cryptographic artifact SHALL be versioned.

Example:

```
identity_commitment_v1  
identity_commitment_v2
```

```
proof_circuit_v1  
proof_circuit_v2
```

```
token_schema_v1  
token_schema_v2
```

The verifier SHALL know which verification procedure applies to each version.

11.9 Emergency Key Revocation

The protocol SHOULD maintain an emergency revocation mechanism independent of the ordinary application deployment pipeline.

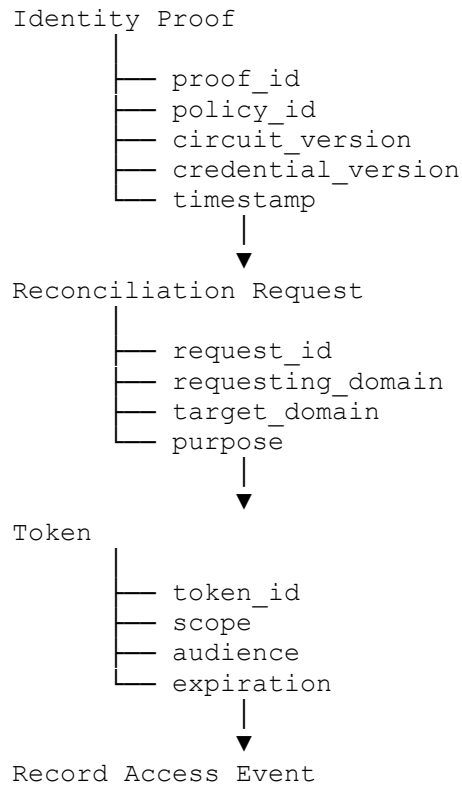
This mechanism MAY be implemented through:

```
Hardware-protected administrative authority  
Dedicated revocation service  
Federated trust registry  
Secure control-plane channel
```

No compromised client application SHALL be able to prevent its own credential from being revoked by the governing trust layer.

11.10 Forensic Event Correlation

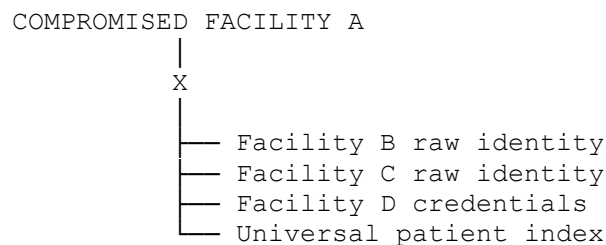
Following an identity-security incident, investigators SHALL be able to correlate:



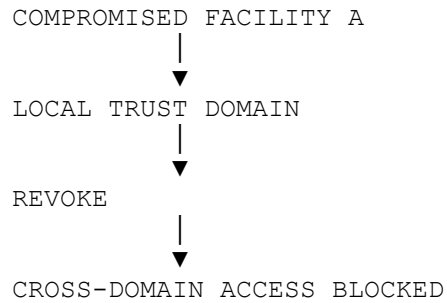
This correlation SHALL be possible without requiring the forensic log to contain the patient's complete plaintext identity.

11.11 Breach Containment Boundary

The architecture SHALL seek to ensure that compromise of one participant does not automatically produce unrestricted access to every other participant.

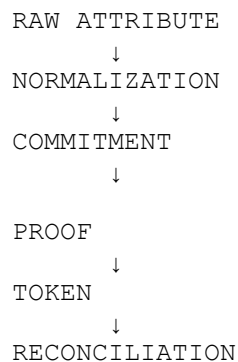


Instead:



11.12 Forensic Identity Exposure Model

The MCIM SHALL evaluate identity exposure as a chain:

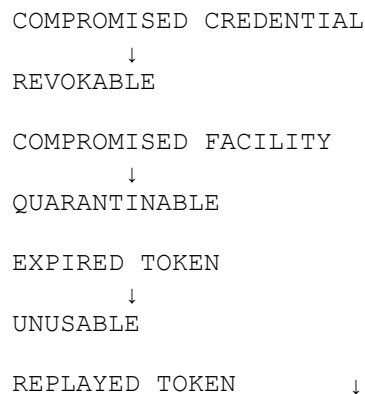


Each stage SHALL expose only the information required for the next stage.

This creates multiple containment boundaries rather than one monolithic identity-access boundary.

11.13 Security Invariant

The identity subsystem SHALL enforce:



REJECTED

REVOKED DOMAIN



BLOCKED

HISTORICAL EVENT



FORENSICALLY TRACEABLE

The MCIM therefore treats compromise containment as a first-class identity requirement rather than assuming that privacy is achieved solely through zero-knowledge proof generation.

PRIVACY-PRESERVING RECORD RECONCILIATION

12.1 Reconciliation Objective

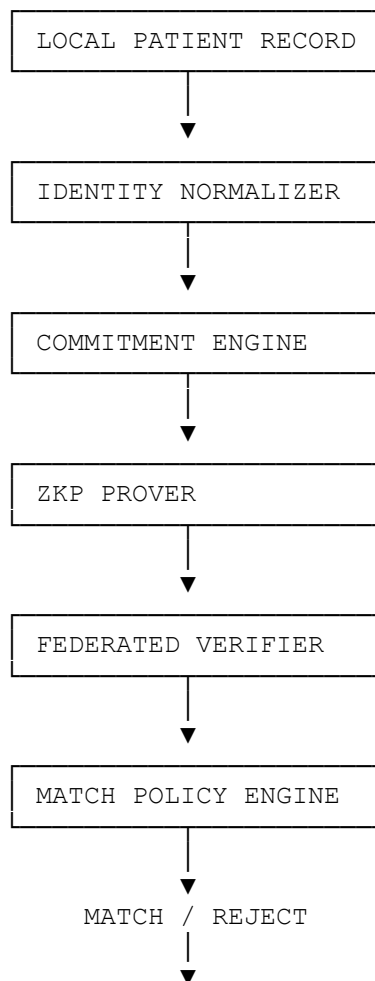
The MCIM SHALL establish a cryptographically authorized relationship between independently held healthcare records without requiring participating systems to construct a shared plaintext patient database.

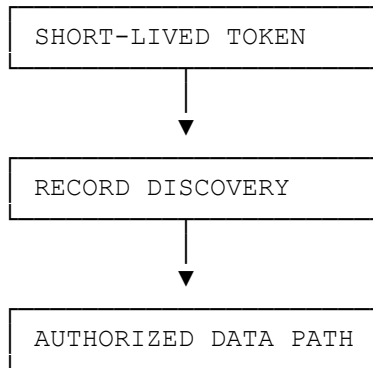
The reconciliation process SHALL distinguish:

IDENTITY MATCH
≠
DATA ACCESS

A successful identity match SHALL therefore not automatically expose the patient's complete clinical history.

12.2 Canonical Reconciliation Flow

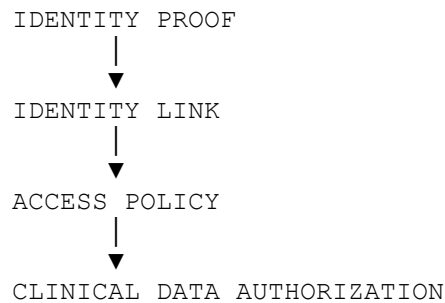




12.3 Separation of Identity and Clinical Data

The identity subsystem SHALL not become a proxy for unrestricted clinical-data access.

The following sequence SHALL therefore remain independent:



A valid identity relationship is a prerequisite for certain workflows, not a universal clinical-data permission.

12.4 Record Discovery

After successful identity reconciliation, the requesting facility MAY receive a minimal record-discovery response.

Example:

```
{
  "status": "MATCH",
  "relationshipId": "opaque-reference",
  "availableResources": [
    "Encounter",
    "Observation",
    "DiagnosticReport"
  ],
  "accessPolicy": "CARE-CONTINUITY-01"
}
```

This response SHALL not automatically contain the underlying clinical records.

12.5 Resource-Level Authorization

Clinical resources SHALL remain independently authorized.

Example:

```
MATCHED PATIENT
├── Encounter      → ALLOWED
├── Observation    → ALLOWED
├── Medication     → RESTRICTED
├── Psychiatric    → RESTRICTED
└── Administrative → POLICY-DEPENDENT
```

The actual categories SHALL be determined by the deployment's applicable legal, clinical, and governance requirements.

12.6 Minimum Necessary Disclosure

Every reconciliation transaction SHALL apply a minimum-disclosure principle.

The protocol SHALL evaluate:

```
WHO
+
WHY
+
WHAT
+
WHERE
+
WHEN
```

before authorizing record access.

The identity match alone SHALL not answer the `WHAT` question.

12.7 Purpose Binding

Every record-reconciliation token SHALL carry a purpose.

Examples:

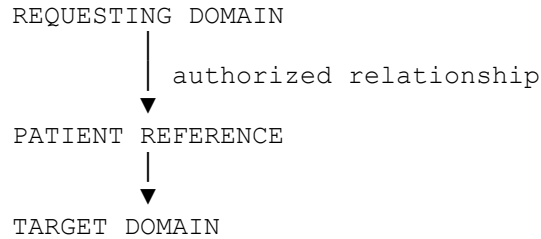
```
CARE_CONTINUITY
REFERRAL
EMERGENCY_CARE
LABORATORY_RECONCILIATION
```

MEDICATION_CONTINUITY
INSURANCE_VERIFICATION

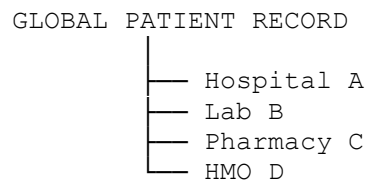
The consuming system SHALL verify that the requested operation is compatible with the declared purpose.

12.8 Cross-Organization Relationship

The protocol SHALL represent the relationship as:



rather than:



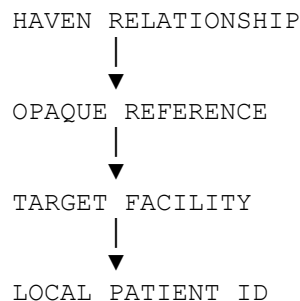
The latter would recreate the centralized identity/data model that the MCIM is specifically designed to avoid.

12.9 Local Identity Resolution

Where possible, the target facility SHALL resolve its own local patient identifier internally.

The Haven protocol SHOULD transmit an opaque relationship reference rather than requiring every participant to understand another participant's database primary keys.

Example:



12.10 No Database Primary-Key Federation

The MCIM SHALL NOT require:

```
Hospital A PatientID  
=  
Hospital B PatientID
```

Instead:

```
Hospital A Local ID  
    |  
    ▼  
Cryptographic Identity Relationship  
    |  
    ▼  
Hospital B Local ID
```

This preserves local database autonomy.

12.11 Record Integrity

A successfully reconciled record SHALL retain its source provenance.

Example:

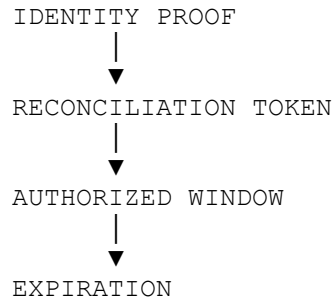
```
{  
  "sourceDomain": "FACILITY-B",  
  "resourceType": "Observation",  
  "resourceReference": "opaque-reference",  
  "identityRelationship": "verified",  
  "provenanceStatus": "SOURCE_AUTHORITY_RETAINED"  
}
```

The reconciliation process SHALL not cause the receiving facility to be mistaken for the original source of the clinical information.

12.12 Identity-Link Expiration

Identity relationships used for active workflow authorization SHALL have bounded validity.

Example:



Expiration SHALL terminate the token's authorization but SHALL not erase legitimate clinical audit records.

ADVERSARIAL THREAT MODEL

13.1 Threat Model Scope

The MCIM SHALL assume that one or more participants may behave maliciously, become compromised, or attempt to extract information beyond their authorized purpose.

The threat model SHALL therefore include:

Malicious Facility
Compromised Adapter
Compromised Credential
Curious Verifier
Compromised Prover
Replay Attacker
Enumeration Attacker
Correlation Attacker
Token Theft
Protocol Downgrade
Policy Manipulation

13.2 Threat T1 — Identity Enumeration

An attacker may attempt to test large numbers of possible identities against cryptographic representations.

Weak construction:

```
SHA256(  
    name +  
    date_of_birth  
)
```

creates significant offline enumeration concerns when the input space is predictable.

MCIM SHALL instead use protected commitment material and appropriate domain separation.

```
SECRET-BOUND COMMITMENT  
    +  
DOMAIN SEPARATION  
    +  
RATE / POLICY CONTROLS
```

13.3 Threat T2 — Cross-Domain Correlation

An attacker may attempt to determine whether the same patient appears across unrelated systems.

The protocol SHALL mitigate this through:

```
Context-specific commitments
Domain separation
Purpose binding
Short-lived relationship tokens
Context-bound nullifiers
```

The architecture SHALL avoid exposing a single universal patient identifier to every participant.

13.4 Threat T3 — Proof Replay

An attacker may capture a valid proof and attempt to reuse it.

Controls:

```
Nonce
+
Context binding
+
Expiration
+
Optional nullifier
+
Single-use token
```

A proof or token outside its authorized context SHALL be rejected.

13.5 Threat T4 — Credential Theft

A stolen credential could permit unauthorized protocol participation.

Response:

```
DETECTION
↓
REVOCATION
↓
DOMAIN / CREDENTIAL QUARANTINE
↓
KEY ROTATION
↓
RE-ENROLLMENT
```

Credential validity SHALL therefore be independently checked during authorization.

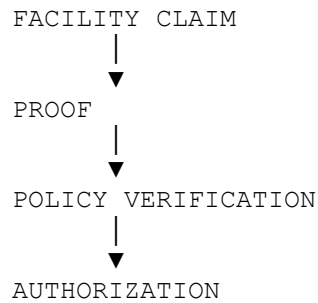
Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

13.6 Threat T5 — Malicious Facility

A participating facility may intentionally submit false identity claims.

The verifier SHALL not trust the assertion merely because the facility is registered.

Instead:



A registered facility SHALL remain subject to credential and policy controls.

13.7 Threat T6 — Compromised Prover

A compromised Prover may attempt to generate fraudulent proofs.

The architecture SHALL therefore bind proofs to:

Authorized credential
Trust domain
Policy
Circuit version
Commitment
Request context

A valid proof from an unauthorized or revoked domain SHALL not automatically authorize reconciliation.

13.8 Threat T7 — Compromised Verifier

A compromised verifier may attempt to request excessive identity relationships.

The architecture SHALL constrain this through:

Purpose binding
Policy restrictions
Audience restrictions
Token scopes
Rate controls
Auditability

The verifier SHALL not receive raw identity attributes simply because it is capable of requesting a proof.

13.9 Threat T8 — Malicious Correlation

Even without raw identity data, repeated observations may create correlation risk.

The MCIM SHALL therefore minimize:

Persistent Identifiers
Unnecessary Metadata
Long-Lived Tokens
Global Correlation Handles
Repeated Unscoped Proofs

Protocol implementations SHOULD avoid emitting unnecessary stable metadata that could become a side-channel identifier.

13.10 Threat T9 — Timing and Metadata Leakage

Cryptographic privacy does not automatically conceal metadata.

Potential leakage includes:

Request timing
Transaction frequency
Facility relationships
Resource availability
Proof frequency
Network topology

Where material to the deployment's threat model, systems SHOULD apply appropriate metadata-minimization techniques.

The MCIM SHALL not claim that zero-knowledge proofs make all network metadata private.

13.11 Threat T10 — Policy Downgrade

An attacker may attempt to replace a strong matching policy with a weaker policy.

Example:

STRONG_ID + DOB
↓
ATTACK
↓
NAME_ONLY

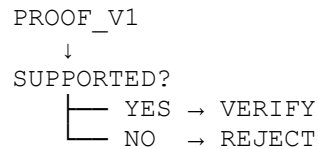
The verifier SHALL reject unauthorized policy substitutions.

Every proof SHALL identify the policy under which it was generated.

13.12 Threat T11 — Cryptographic Downgrade

The same principle SHALL apply to cryptographic versions.

A verifier SHALL reject unsupported or deprecated proof systems unless the active governance policy explicitly authorizes them.



13.13 Threat T12 — Insider Abuse

An authorized operator may attempt to use legitimate privileges for an unauthorized purpose.

Controls SHALL include:

- Purpose binding
- Least privilege
- Scoped tokens
- Audit trails
- Credential separation
- Revocation
- Administrative review

Cryptography SHALL therefore be combined with governance controls.

13.14 Threat T13 — Database Compromise

If a participant's database is compromised, the attacker may obtain local identity information.

MCIM does not make local database compromise impossible.

Instead, its containment objective is:

```
COMPROMISED LOCAL DATABASE
      ≠
AUTOMATIC ACCESS TO OTHER
PARTICIPATING DATABASES
```

The federation boundary SHALL prevent a local compromise from becoming a universal identity breach.

13.15 Threat T14 — Unauthorized Identity Discovery

The protocol SHALL reject requests that cannot establish:

```
Authorized Purpose
+
Authorized Trust Relationship
+
Valid Credential
+
Valid Proof
+
Valid Policy
```

This converts identity discovery from an open query capability into a controlled protocol operation.

13.16 Threat-Control Matrix

Threat	Primary Control
Enumeration	Secret-bound commitments
Cross-domain correlation	Domain separation
Proof replay	Nonce + context + expiration
Credential theft	Revocation + rotation
Malicious facility	Proof + policy verification
Compromised prover	Credential-bound proof
Compromised verifier	Scoped authorization
Metadata correlation	Metadata minimization
Policy downgrade	Policy/version binding
Cryptographic downgrade	Version enforcement
Insider abuse	Purpose + least privilege
Database compromise	Federated isolation

13.17 Forensic Security Conclusion

The MCIM's security model is therefore based on **compartmentalization rather than absolute trust**.

IDENTITY
↓
LOCALIZED

PROOF
↓
MINIMIZED

AUTHORIZATION
↓
SCOPED

TOKEN
↓
SHORT-LIVED

CREDENTIAL
↓
REVOCABLE

TRUST DOMAIN
↓
ISOLATABLE

AUDIT
↓
FORENSICALLY CORRELATABLE

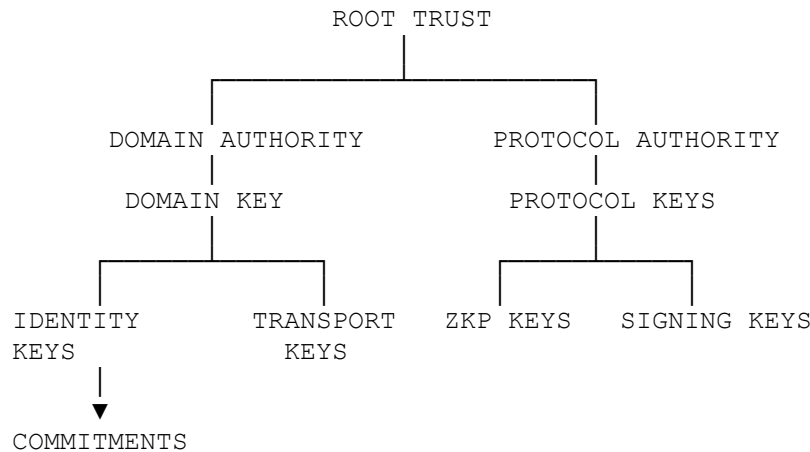
The architecture consequently reduces the blast radius of identity compromise while preserving the ability to establish authorized relationships across independently governed healthcare networks.

CRYPTOGRAPHIC KEY HIERARCHY AND CUSTODY

14.1 Key Hierarchy

The MCIM SHALL maintain cryptographic separation between institutional, identity, proof, transport, and authorization functions.

Reference hierarchy:



No single application key SHALL be permitted to perform every cryptographic function.

14.2 Functional Key Separation

At minimum, the deployment SHOULD separate:

id="q7m3xr"

K_DOMAIN
Trust-domain identity

K_IDENTITY
Identity commitment operations

K_TRANSPORT
Secure service communication

K_PROVER
ZKP proof generation authorization

K_SIGNING
Token / protocol signing

K_AUDIT
Audit integrity operations

Compromise of one key SHALL therefore not automatically compromise every cryptographic function.

14.3 Root Trust Material

Root trust material SHALL have the highest protection level.

It SHOULD be maintained within:

Hardware Security Module
Air-gapped signing environment
Hardware-backed secure enclave

The root key SHOULD NOT be exposed to ordinary application processes.

14.4 Domain Key

Each participating organization SHALL have a domain cryptographic identity.

Conceptual representation:

```
DOMAIN_ID
+
PUBLIC_KEY
+
CREDENTIAL
+
POLICY_STATUS
```

The corresponding private key SHALL remain under the control of that organization.

14.5 Identity Commitment Key

The commitment layer SHALL use dedicated key material where keyed commitments are employed.

Conceptually:

```
C = Commit(
    normalized_identity,
    secret,
    domain_separator
)
```

The commitment SHALL be deterministic within the intended matching context while remaining resistant to unauthorized offline identity testing.

14.6 Domain Separation

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

The same identity material SHALL not generate an identical reusable commitment across unrelated contexts.

Conceptual construction:

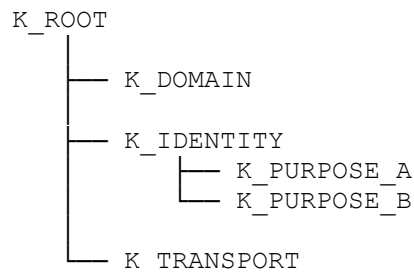
```
DOMAIN_SEPARATOR =  
    protocol_id  
    ||  
    jurisdiction  
    ||  
    trust_domain  
    ||  
    purpose  
    ||  
    commitment_version
```

This limits cross-context correlation.

14.7 Key Derivation

Where hierarchical derivation is used, a hardened derivation mechanism SHALL be employed.

Conceptual:

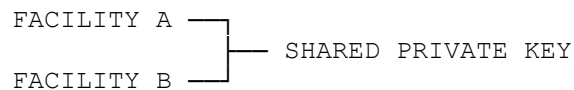


Derived keys SHALL inherit the security assumptions of the parent key.

14.8 No Shared Private Keys

Two independent trust domains SHALL NOT share one private cryptographic key.

Incorrect:



Correct:

```
FACILITY A → K_A  
FACILITY B → K_B
```

Federation SHALL be established through verification of public credentials rather than shared private-key custody.

14.9 Hardware-Backed Custody

Where hardware-backed custody is available, private keys SHALL be marked non-exportable where technically feasible.

The application should receive:

```
SIGN()  
VERIFY()  
DERIVE()
```

operations rather than raw private-key material.

14.10 Token Signing Key

The token issuer SHALL maintain a dedicated signing key.

The verifier SHALL obtain the corresponding trusted public key through the federation trust mechanism.

Token verification SHALL include:

```
Signature  
Issuer  
Audience  
Policy  
Expiration  
Revocation
```

14.11 ZKP Proving Keys

Where the selected ZKP system requires a proving key, the proving-key lifecycle SHALL be governed independently from patient identity keys.

The proving key SHALL correspond to a specific:

```
Circuit Version  
Constraint System  
Protocol Version  
Parameter Set
```

14.12 ZKP Verification Keys

Verification keys SHALL be versioned and distributed through an authenticated trust mechanism.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

Example:

```
{
  "proofSystem": "ZKP-V1",
  "circuitId": "MCIM-IDENTITY-MATCH",
  "circuitVersion": "1.0.0",
  "verificationKeyId": "VK-2026-01",
  "status": "ACTIVE"
}
```

14.13 Key Status

Every production key SHALL have a lifecycle status:

GENERATED
PENDING
ACTIVE
ROTATION_PENDING
REVOKED
COMPROMISED
DESTROYED

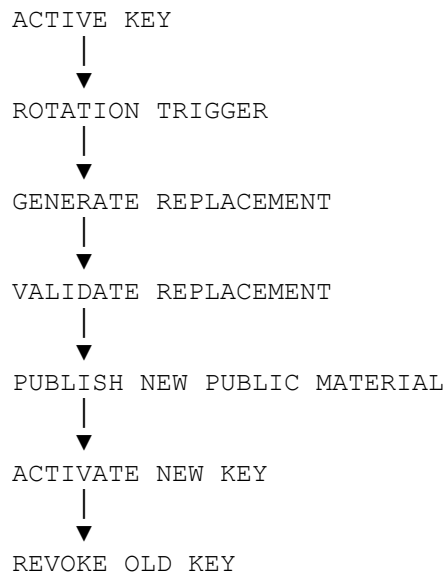
A key marked REVOKED or COMPROMISED SHALL not authorize new protocol operations.

KEY ROTATION AND CRYPTOGRAPHIC DISASTER RECOVERY

15.1 Rotation Objective

Key rotation SHALL prevent long-lived cryptographic material from becoming a permanent security dependency.

The standard lifecycle SHALL be:



15.2 Rotation Triggers

Rotation SHALL occur according to defined policy.

Triggers MAY include:

- Scheduled lifecycle expiration
- Suspected compromise
- Confirmed compromise
- Personnel / administrative change
- Cryptographic algorithm deprecation
- Proof-circuit upgrade
- Jurisdictional requirement
- Security incident

15.3 Overlapping Rotation Window

For non-emergency rotation, the system MAY temporarily support:

OLD KEY → VERIFY ONLY
NEW KEY → SIGN / GENERATE

This permits existing valid artifacts to remain verifiable while preventing new artifacts from being generated under the retiring key.

15.4 Emergency Rotation

Emergency rotation SHALL compress the lifecycle:

```
COMPROMISE
  ↓
REVOKE
  ↓
QUARANTINE
  ↓
GENERATE NEW KEY
  ↓
RE-ENROLL
  ↓
REVALIDATE
```

The compromised key SHALL not remain active merely to preserve operational convenience.

15.5 Commitment Version Migration

When commitment keys or commitment algorithms change:

```
COMMITMENT_V1
  |
  ▼
LEGACY / READ-ONLY
  |
  ▼
COMMITMENT_V2
  |
  ▼
NEW OPERATIONS
```

The system SHALL retain sufficient metadata to determine which verification process applies to historical records.

15.6 Identity Reconciliation During Migration

A migration SHALL not silently reinterpret historical identity relationships.

Each relationship SHALL retain:

```
identity_version
commitment_version
policy_version
proof_version
credential_version
```

This permits forensic reconstruction of the exact cryptographic environment under which the relationship was established.

15.7 ZKP Circuit Upgrade

ZKP circuits SHALL be immutable by version.

Example:

```
MCIM-IDENTITY-MATCH-V1
  |
  ▼
MCIM-IDENTITY-MATCH-V2
```

V2 SHALL not overwrite the definition of V1.

Historical V1 proofs SHALL remain identifiable as V1 artifacts.

15.8 Verification-Key Transition

During an approved transition:

```
VERIFIER
├── VK-V1 → historical verification
└── VK-V2 → current verification
```

The verifier SHALL reject verification keys that are neither trusted nor associated with the declared proof version.

15.9 Cryptographic Backup

Backups SHALL contain sufficient material to restore cryptographic service without exposing private keys unnecessarily.

Backup architecture SHOULD separate:

PUBLIC MATERIAL
+
CONFIGURATION
+
POLICY VERSIONS
+
AUDIT METADATA

from:

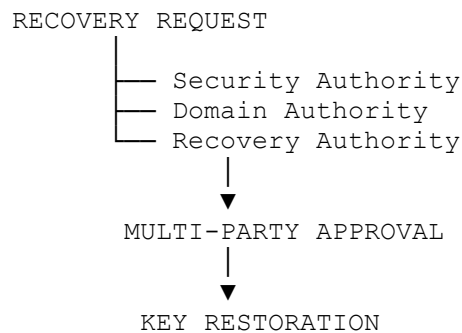
PRIVATE KEY MATERIAL

Private keys SHOULD remain within protected hardware or encrypted key-recovery infrastructure.

15.10 Key Recovery

Where recovery is necessary, the system SHALL require multi-party authorization.

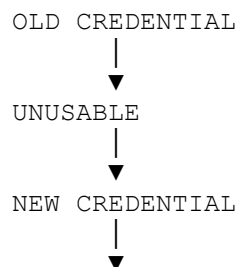
Conceptually:



No single ordinary application operator SHOULD possess sufficient authority to recover protected root credentials.

15.11 Loss of Key Material

If an unrecoverable private key is lost:



RE-ENROLLMENT

The system SHALL not fabricate replacement signatures for historical events.

Historical records SHALL preserve the fact that the original credential is no longer available for live signing.

15.12 Disaster Recovery States

The cryptographic control plane SHALL define:

NORMAL
DEGRADED
QUARANTINED
RECOVERY
RESTORED

During `QUARANTINED` state, new identity-link operations SHALL fail closed unless an explicitly governed emergency procedure exists.

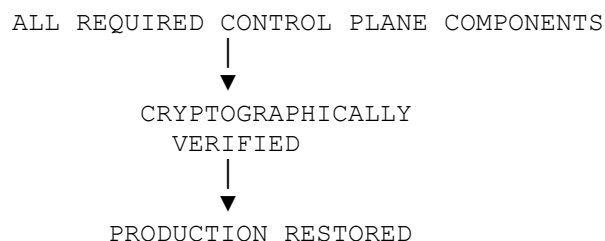
15.13 Recovery Integrity Verification

After restoration, the following SHALL be verified before returning to production:

Trust Registry Integrity
+
Credential Registry Integrity
+
Key Version Integrity
+
Policy Registry Integrity
+
ZKP Verification-Key Integrity
+
Audit-Chain Integrity
+
Clock / Timestamp Integrity

15.14 Recovery Completion

The recovery sequence SHALL terminate only when:



A partially restored cryptographic environment SHALL not silently resume normal identity reconciliation.

15.15 Final Key-Custody Invariant

The MCIM SHALL preserve the following architectural separation:

PATIENT IDENTITY
≠
PRIVATE KEY

PRIVATE KEY
≠
TOKEN

TOKEN
≠
CLINICAL RECORD

CLINICAL RECORD
≠
FEDERATED TRUST REGISTRY

This separation prevents any single artifact from becoming a universal credential capable of reconstructing or controlling the entire multi-party identity graph.

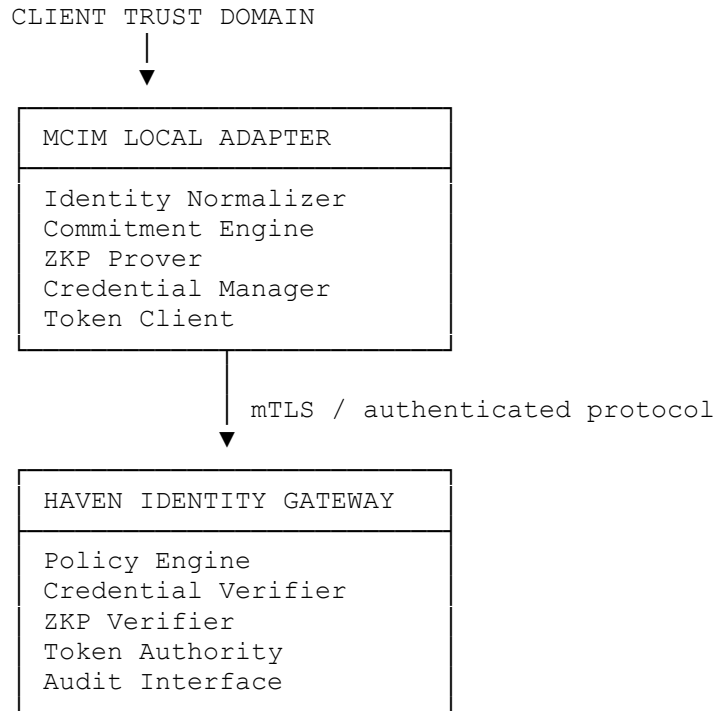
The cryptographic identity architecture therefore remains **distributed, versioned, revocable, recoverable, and independently auditable** throughout the operational lifecycle.

MCIM PROTOCOL AND API INTERFACE SPECIFICATION

16.1 Protocol Boundary

The MCIM SHALL expose a cryptographic coordination interface rather than a centralized patient database interface.

The primary protocol boundary SHALL be:



The API SHALL NOT require the client to upload a complete plaintext patient identity record merely to perform an identity-link operation.

16.2 Identity Match Request

A canonical request MAY use:

```
{
  "protocolVersion": "MCIM/1.0",
  "requestId": "req-opaque-uuid",
  "requestingDomain": "DOMAIN-A",
  "targetDomain": "DOMAIN-B",
  "policyId": "NG-GB-CARE-01",
  "purpose": "CARE_CONTINUITY",
  "proof": {
    "proofSystem": "ZKP-V1",
    "circuitId": "MCIM-IDENTITY-MATCH",
    "circuitVersion": "1.0.0",
```

```

    "proofData": "encoded-proof"
  },
  "commitment": {
    "scheme": "BOUND-COMMITMENT-V1",
    "value": "encoded-commitment"
  },
  "nonce": "unique-request-nonce"
}

```

The exact serialization SHALL be governed by the deployed protocol version.

16.3 Request Validation

The receiving gateway SHALL validate:

```

protocolVersion
requestId
requestingDomain
targetDomain
policyId
purpose
proofSystem
circuitVersion
proofData
commitment
nonce

```

before attempting authorization.

Malformed requests SHALL terminate before identity matching.

16.4 ZKP Verification Interface

The verifier SHALL conceptually expose:

```

def verify_identity_proof(
    proof,
    public_inputs,
    verification_key
) -> bool:
    ...

```

The verification operation SHALL validate the cryptographic proof against the exact circuit and verification-key version declared by the request.

A valid proof under a different circuit SHALL not be accepted as equivalent.

16.5 Public Inputs

Public inputs SHALL be minimized.

Depending upon policy, they MAY contain:

```
policy_id
purpose
domain identifiers
commitment
nonce
circuit version
context hash
```

They SHOULD NOT contain unnecessary patient-identifying attributes.

16.6 Proof Response

A successful response MAY return:

```
{
  "status": "MATCH",
  "requestId": "req-opaque-uuid",
  "relationshipId": "opaque-relationship-reference",
  "policyId": "NG-GB-CARE-01",
  "proofVerified": true,
  "token": {
    "tokenId": "opaque-token-id",
    "expiresAt": "2026-08-30T12:55:00Z",
    "audience": "DOMAIN-B",
    "purpose": "CARE_CONTINUITY"
  }
}
```

The response SHALL avoid returning raw source identity attributes unless a separate policy explicitly permits their disclosure.

16.7 No-Match Response

A no-match response SHALL reveal only the minimum information required.

Example:

```
{
  "status": "NO_MATCH",
  "requestId": "req-opaque-uuid",
  "reasonCode": "IDENTITY_PREDICATE_FAILED"
}
```

The service SHOULD avoid disclosing which individual attribute caused the failure where doing so could facilitate identity enumeration.

16.8 Ambiguous Response

Ambiguous results SHALL be explicit:

```
{
  "status": "AMBIGUOUS",
  "requestId": "req-opaque-uuid",
  "reasonCode": "MULTIPLE_CANDIDATES"
}
```

An ambiguous result SHALL not automatically produce an identity-link token.

16.9 Protocol Error Classes

The MCIM SHALL maintain machine-readable error codes.

MCIM-AUTH-001
UNTRUSTED_DOMAIN

MCIM-AUTH-002
CREDENTIAL_REVOKED

MCIM-POL-001
POLICY_INACTIVE

MCIM-POL-002
PURPOSE_NOT_PERMITTED

MCIM-ZKP-001
INVALID_PROOF

MCIM-ZKP-002
UNSUPPORTED_CIRCUIT

MCIM-ID-001
IDENTITY_PREDICATE_FAILED

MCIM-ID-002
AMBIGUOUS_MATCH

MCIM-TKN-001
TOKEN_EXPIRED

MCIM-TKN-002
TOKEN_REPLAY

MCIM-TKN-003
TOKEN_REVOKED

MCIM-REQ-001

MALFORMED_REQUEST

16.10 Fail-Closed Interface Behavior

Unknown errors SHALL not cause the gateway to authorize the request.

Conceptually:

```
try:
    result = process_request(request)
except Exception:
    result = "FAIL_CLOSED"
```

Production implementations SHALL use explicit exception classes and controlled error handling rather than broad exception suppression.

16.11 Transport Security

Protocol communication SHALL use authenticated encrypted transport.

Recommended baseline:

```
TLS 1.3
+
Mutual TLS
+
Certificate / Credential Validation
+
Short-Lived Credentials
```

Where mTLS is unsuitable for a particular deployment, an equivalent cryptographically authenticated transport SHALL be specified explicitly.

MCIM IMPLEMENTATION CONTRACT AND INTEROPERABILITY

17.1 Implementation Contract

Every MCIM implementation SHALL conform to a defined interoperability contract.

The contract SHALL specify:

- Protocol Version
- Message Schema
- Proof System
- Circuit Version
- Commitment Scheme
- Credential Format
- Token Format
- Policy Identifier
- Error Model
- Audit Requirements

An implementation SHALL not claim interoperability merely because it can exchange HTTP requests.

17.2 Version Negotiation

Protocol participants SHALL explicitly negotiate or declare supported versions.

Example:

```
{
  "protocol": "MCIM",
  "supportedVersions": [
    "1.0"
  ],
  "proofSystems": [
    "ZKP-V1"
  ]
}
```

Unsupported protocol versions SHALL result in controlled rejection.

17.3 Capability Discovery

A trust domain MAY expose a capability descriptor containing:

- supported protocol versions
- supported proof systems
- supported circuits
- supported credential versions
- supported policy classes
- supported token schemas

Capability discovery SHALL not expose sensitive patient information.

17.4 Schema Validation

Incoming messages SHALL be validated against the active schema before processing.

Conceptually:

```
def validate_message(message, schema):  
  
    validate(  
        instance=message,  
        schema=schema  
    )  
  
    return True
```

Invalid schema instances SHALL fail closed.

17.5 Canonical Serialization

Cryptographic payloads SHALL use deterministic serialization where byte-level equality is required.

The implementation SHALL define:

```
Field ordering  
Encoding  
Unicode normalization  
Numeric representation  
Timestamp representation  
Null handling  
Array ordering
```

Ambiguous serialization SHALL not be used as cryptographic input.

17.6 Hashing

When payload hashing is required, the implementation SHALL hash a canonical representation.

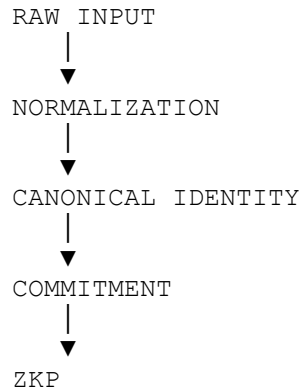
Conceptual:

```
canonical = canonicalize(payload)  
  
digest = sha256(  
    canonical.encode("utf-8")  
) .hexdigest()
```

The protocol SHALL record the hash algorithm and canonicalization version.

17.7 Identity Normalization Boundary

Normalization SHALL occur before cryptographic commitment generation.



The same logical identity represented differently by two source systems SHALL not produce incompatible commitments merely because of formatting differences that the active policy defines as equivalent.

17.8 Normalization Rules

Implementations SHALL define deterministic rules for:

- Unicode normalization
- Whitespace
- Case normalization
- Telephone formatting
- Date representation
- Approved name transliteration
- Identifier formatting
- Null / missing fields

Rules SHALL be versioned.

A normalization rule change SHALL create a new version rather than silently modifying historical semantics.

17.9 Cryptographic Payload Boundary

The payload transmitted between domains SHOULD resemble:

```
{
  "version": "1.0",
  "domain": "DOMAIN-A",
  "policy": "CARE-CONTINUITY-01",
  "purpose": "CARE_CONTINUITY",
  "commitment": "opaque-value",
  "proof": "opaque-value",
```

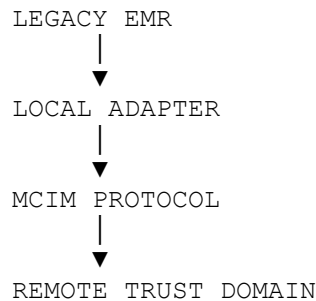
Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

```
"nonce": "opaque-value",  
"contextHash": "opaque-value"  
}
```

The payload intentionally represents a **proof of an authorized relationship**, not a portable plaintext patient record.

17.10 Interoperability With Legacy Systems

The MCIM SHALL permit legacy systems to remain internally autonomous.



The remote facility SHALL not need direct access to the legacy database engine.

17.11 Database Independence

The adapter SHOULD support heterogeneous storage engines, including:

SQL
PostgreSQL
MySQL
Microsoft SQL Server
Oracle Database

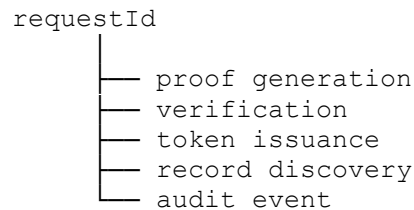
NoSQL
MongoDB
Cassandra
Document Stores
Key-Value Stores

The protocol SHALL operate on an abstract canonical identity representation rather than on database-specific query semantics.

17.12 Transaction Correlation

Each identity-link operation SHALL have a unique correlation identifier.

Example:



This allows forensic reconstruction of the protocol transaction.

17.13 Clock Requirements

Because token expiration and replay prevention depend upon time, participating systems SHALL maintain sufficiently synchronized clocks.

The implementation SHOULD use authenticated or otherwise trusted time synchronization.

Significant clock drift SHALL cause token validation to fail closed where the drift could compromise security.

17.14 Observability

Operational telemetry SHALL measure:

- Proof verification latency
- Match decisions
- Token issuance failures
- Credential failures
- Protocol errors
- Circuit versions
- Policy versions
- Replay attempts
- Revocation events

Telemetry SHALL avoid unnecessary patient-identifying information.

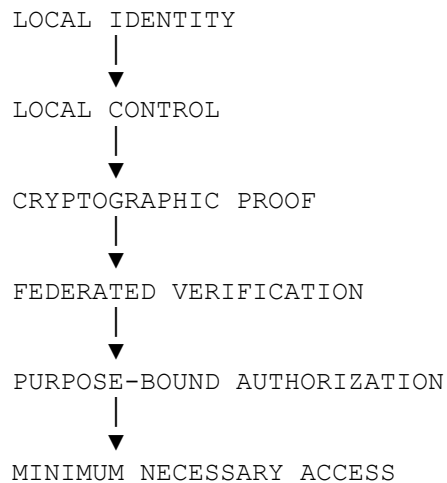
17.15 Interoperability Acceptance Test

A new implementation SHALL pass at minimum:

- [] Schema validation
- [] Canonicalization tests
- [] Commitment determinism tests
- [] ZKP verification tests
- [] Invalid-proof rejection
- [] Replay rejection
- [] Expiration rejection
- [] Revocation rejection
- [] Purpose-binding tests
- [] Scope enforcement
- [] Cross-version rejection
- [] Audit correlation
- [] Fail-closed behavior

17.16 Protocol Conformance Rule

An implementation SHALL be considered MCIM-conformant only when it preserves the following invariant:



Any implementation that bypasses the cryptographic authorization boundary and directly exports unrestricted identity data SHALL be considered **non-conformant with the privacy-preserving MCIM architecture**.

17.17 Final Interoperability Principle

MCIM interoperability SHALL therefore mean:

DIFFERENT DATABASES
+
DIFFERENT IDENTIFIERS
+
DIFFERENT JURISDICTIONS
+
DIFFERENT HEALTHCARE SYSTEMS
↓
COMMON CRYPTOGRAPHIC
COORDINATION CONTRACT

The protocol standardizes the **trust relationship and cryptographic evidence**, not ownership of the underlying healthcare databases.

This preserves the fundamental architectural property of Haven: **distributed clinical identity custody with interoperable cryptographic coordination**.

FORMAL SECURITY INVARIANTS AND COMPLIANCE CONTROL MODEL

18.1 Identity Security Invariants

The MCIM SHALL preserve the following invariants throughout the complete identity lifecycle:

INVARIANT 01

No raw identity attribute is required
for routine cross-domain proof verification.

INVARIANT 02

No identity proof alone grants clinical-data access.

INVARIANT 03

No token survives beyond its authorized lifetime.

INVARIANT 04

No revoked credential may authorize new operations.

INVARIANT 05

No ambiguous identity match automatically becomes
a verified identity relationship.

INVARIANT 06

No participating trust domain receives unrestricted
authority over another domain's patient database.

INVARIANT 07

No proof generated under an unauthorized policy
is accepted as a valid production identity assertion.

18.2 End-to-End Authorization Predicate

The final identity-link authorization decision SHALL be represented conceptually as:

```
AUTHORIZED_LINK =  
    TRUST_DOMAIN_VALID  
    ^ CREDENTIAL_VALID  
    ^ POLICY_ACTIVE  
    ^ PURPOSE_ALLOWED  
    ^ PROOF_VALID  
    ^ COMMITMENT_VALID  
    ^ CONTEXT_VALID  
    ^ TOKEN_VALID  
    ^ TOKEN_NOT_REVOKED  
    ^ TOKEN_NOT_REPLAYED  
    ^ SCOPE_ALLOWED
```

If any mandatory predicate evaluates to false:

```
AUTHORIZED_LINK = FALSE
```

The system SHALL fail closed.

18.3 Identity Match vs Authorization

The MCIM SHALL explicitly separate two decisions:

DECISION A

"Do the authorized identity predicates establish a relationship?"

≠

DECISION B

"Is this requesting party authorized to access this resource for this purpose?"

A successful Decision A SHALL not imply a successful Decision B.

18.4 Clinical Data Authorization

The clinical data layer SHALL independently evaluate:

```
Requester
+
Resource
+
Purpose
+
Jurisdiction
+
Consent / Legal Basis where applicable
+
Policy
+
Token Scope
```

The identity subsystem SHALL not silently override these controls.

18.5 Privacy-Control Mapping

The MCIM SHALL provide technical controls supporting common privacy principles.

Privacy Principle	MCIM Technical Control
Data minimization	Proof / commitment exchange
Purpose limitation	Purpose-bound tokens
Access control	Scoped authorization
Storage limitation	Short-lived tokens
Integrity	Cryptographic signatures / hashes
Confidentiality	Encrypted transport + protected credentials
Accountability	Correlated audit events
Data localization	Local identity custody
Least privilege	Resource / scope restrictions
Revocation	Credential and token revocation

These controls are technical mechanisms, not a legal determination that a deployment is compliant with every applicable regulation.

18.6 Data Protection Boundary

The MCIM SHALL define three principal data classes:

CLASS A
RAW IDENTITY DATA
Retained by originating trust domain.

CLASS B
CRYPTOGRAPHIC IDENTITY ARTIFACTS
Commitments, proofs, nullifiers, and protocol metadata.

CLASS C
AUTHORIZED CLINICAL REFERENCES
Opaque references and policy-scoped resource identifiers.

The protocol SHOULD prevent unnecessary movement of Class A information.

18.7 Auditability

Every material identity operation SHOULD generate a correlated audit event.

Example:

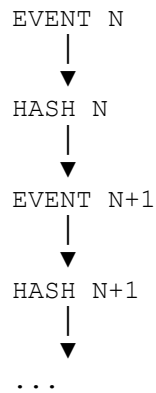
```
{
  "eventType": "IDENTITY_RECONCILIATION",
  "requestId": "opaque-request-id",
  "proofId": "opaque-proof-id",
  "policyId": "CARE-CONTINUITY-01",
  "credentialVersion": "3",
  "circuitVersion": "1.0.0",
  "decision": "MATCH",
  "timestamp": "2026-08-30T12:45:00Z"
}
```

Audit records SHALL minimize patient-identifying content.

18.8 Tamper Evidence

Audit records SHOULD be protected through cryptographic integrity mechanisms.

Conceptually:



Where an append-only or cryptographically chained audit architecture is used, modification of historical entries SHALL become detectable.

18.9 Compliance Evidence

The system SHALL be capable of producing evidence showing:

WHO initiated the operation
WHAT policy authorized it
WHICH credential was used
WHICH proof version was verified
WHEN authorization occurred

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

WHICH scope was granted
WHEN the token expired
WHETHER revocation occurred
WHICH resource relationship was accessed

The evidence SHALL be generated without requiring investigators to reconstruct the entire patient identity from cryptographic logs.

18.10 Data Subject Rights Architecture

Where applicable, deployments SHALL map data-subject rights to the actual authoritative system holding the relevant data.

The MCIM SHALL not assume that because it stores a cryptographic relationship artifact, it consequently owns or controls the underlying clinical record.

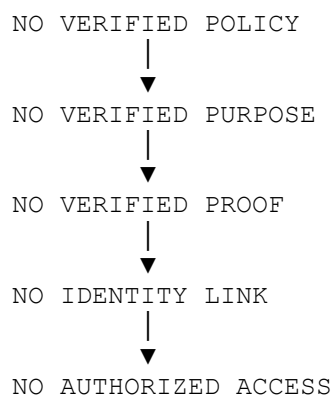
Requests concerning:

Correction
Deletion
Access
Restriction
Portability

SHALL be routed according to the applicable jurisdictional governance model and authoritative data controller / custodian structure.

18.11 Privacy Failure State

If the system cannot establish the required privacy and authorization conditions:



There SHALL be no automatic fallback to unrestricted plaintext identity exchange.

MCIM DEPLOYMENT ACCEPTANCE AND FINAL FORENSIC CONCLUSION

19.1 Production Acceptance Criteria

An MCIM deployment SHALL not be considered production-ready until the following conditions have been validated:

CRYPTOGRAPHIC

- ☐ Approved proof system implemented
- ☐ Verification keys authenticated
- ☐ Commitment scheme versioned
- ☐ Key custody validated
- ☐ Rotation procedures tested

IDENTITY

- ☐ Canonicalization deterministic
- ☐ Matching predicates policy-defined
- ☐ Ambiguous matches fail closed
- ☐ Collision handling tested
- ☐ Cross-domain correlation controls tested

AUTHORIZATION

- ☐ Purpose binding enforced
- ☐ Token scope enforced
- ☐ Audience enforced
- ☐ Expiration enforced
- ☐ Replay protection verified
- ☐ Revocation verified

FEDERATION

- ☐ Trust-domain enrollment validated
- ☐ Jurisdiction policy active
- ☐ Credential lifecycle tested
- ☐ Domain quarantine tested

SECURITY

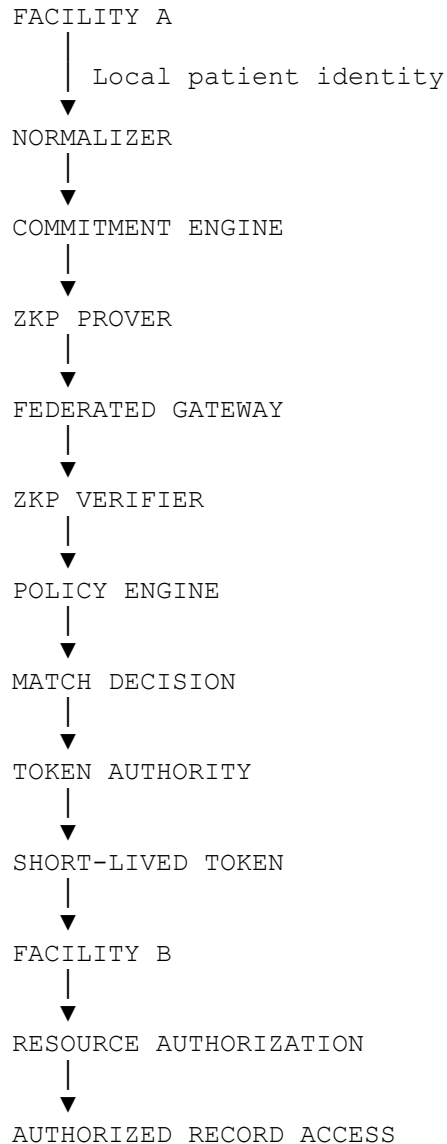
- ☐ Enumeration resistance assessed
- ☐ Metadata leakage assessed
- ☐ Compromise response tested
- ☐ Disaster recovery tested
- ☐ Audit integrity verified

INTEROPERABILITY

- ☐ Schema validation passed
- ☐ Version negotiation passed
- ☐ Canonical serialization passed
- ☐ Legacy adapter integration passed
- ☐ End-to-end reconciliation passed

19.2 Identity-Reconciliation Acceptance Test

The minimum end-to-end test SHALL demonstrate:



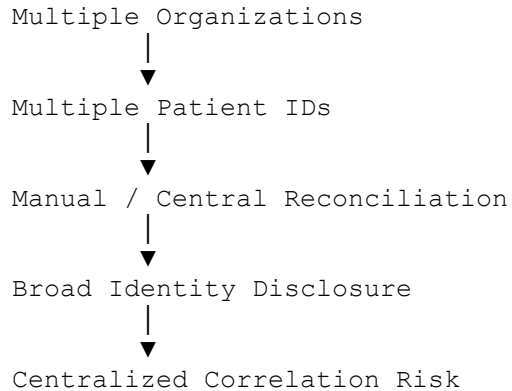
The test SHALL additionally demonstrate that:

Invalid proof	→ REJECTED
Expired token	→ REJECTED
Revoked credential	→ REJECTED
Wrong purpose	→ REJECTED
Wrong audience	→ REJECTED
Replay	→ REJECTED
Ambiguous identity	→ NO AUTOMATIC LINK
Unsupported version	→ REJECTED

19.3 Forensic Identity Crisis Transformation

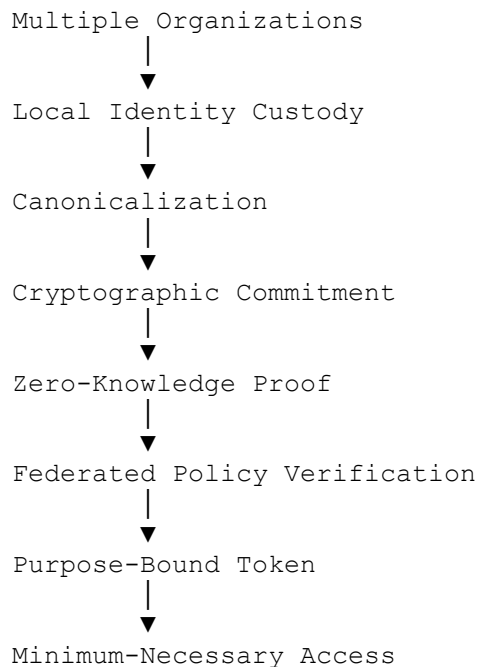
The MCIM changes the fundamental architecture of multi-party identity coordination:

BEFORE



becomes:

AFTER



19.4 What the MCIM Actually Eliminates

The architecture can eliminate or substantially reduce the requirement for:

A universal plaintext patient identifier
A centralized plaintext identity repository

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

Routine cross-border identity replication
Unscoped identity assertions
Permanent reconciliation credentials
Direct federation of legacy database primary keys

It does **not** eliminate:

Local database compromise
Human identity errors
Incorrect source records
Regulatory obligations
Clinical governance
Consent / legal-basis requirements
Metadata leakage
Endpoint compromise
Operational misconfiguration

These remain outside the cryptographic proof system and require independent controls.

19.5 Final Security Proposition

The MCIM's core proposition is therefore:

DO NOT CENTRALIZE THE IDENTITY.

DO NOT REPLICATE THE IDENTITY.

DO NOT TRUST THE ASSERTION ALONE.

PROVE THE REQUIRED RELATIONSHIP.

AUTHORIZE THE REQUIRED PURPOSE.

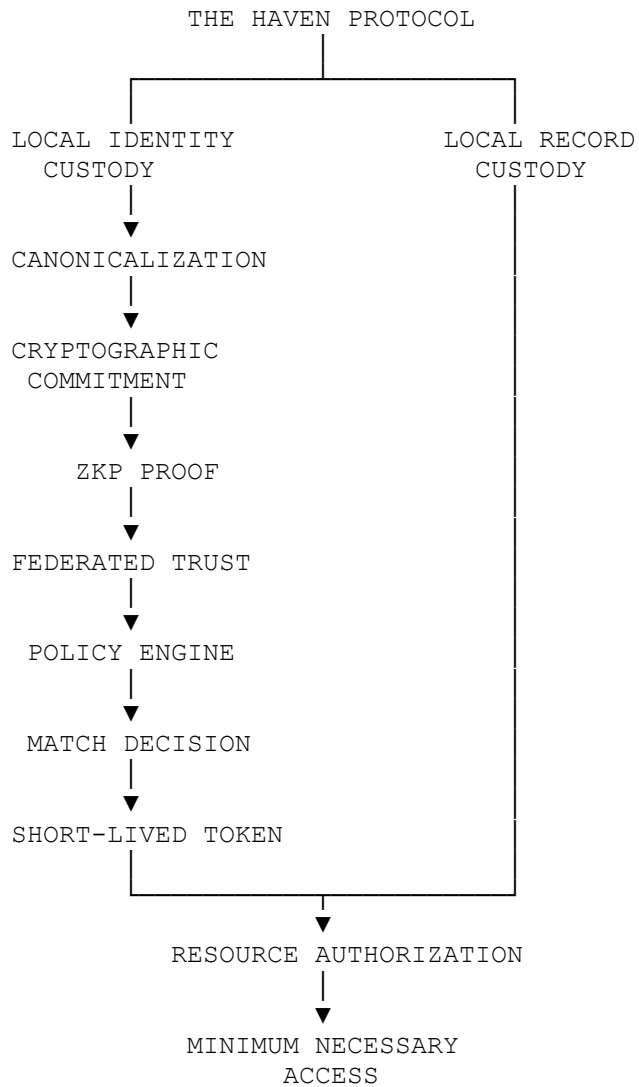
DISCLOSE ONLY WHAT IS NECESSARY.

EXPIRE THE AUTHORIZATION.

REVOKE THE CREDENTIAL.

AUDIT THE EVENT.

19.6 Final Architectural Model

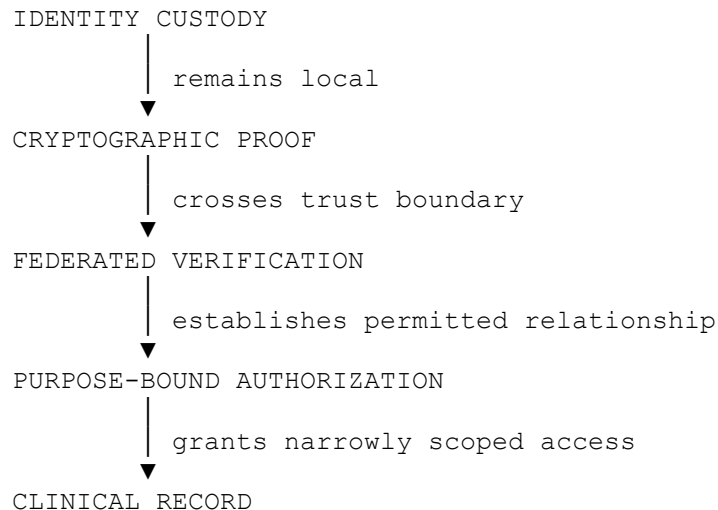


19.7 Final Forensic Conclusion

The Multi-Party Cryptographic Identity Matrix does not attempt to solve healthcare fragmentation by constructing another centralized identity authority.

It solves the identity coordination problem by introducing a **cryptographically verifiable relationship layer between independently controlled identity domains**.

The decisive architectural boundary is:



Consequently, Haven does not need to own every identity, database, or patient record in order to coordinate continuity between them.

The protocol's security objective is not to make identity risk mathematically nonexistent. Its objective is to **remove unnecessary identity concentration, constrain cross-domain linkability, make authorization cryptographically verifiable, make credentials revocable, make access purpose-bound, and make material protocol events forensically reconstructable**.

That distinction is fundamental.

The resulting system is therefore best characterized as:

DISTRIBUTED IDENTITY CUSTODY
+
ZERO-KNOWLEDGE RECONCILIATION
+
FEDERATED TRUST
+
DETERMINISTIC MATCHING
+
PURPOSE-BOUND AUTHORIZATION
+
SHORT-LIVED CRYPTOGRAPHIC TOKENS
+
REVOCABLE CREDENTIALS
+
FORENSIC AUDITABILITY

This constitutes the final architectural security boundary of the **Haven Protocol Multi-Party Cryptographic Identity Matrix (MCIM v2.0)**.